4.01-hpi-unos-example

September 2, 2024

1 PRiSM Example: UNOS Heart Transplant Data

This notebook demonstrates the PRiSM (Partial Responses in Structured Models) method, which is designed to transform black-box classifiers into inherently interpretable models without compromising predictive performance. This approach is particularly valuable in high-stakes applications where understanding the model's decision-making process is crucial.

The method addresses a key challenge in machine learning: the trade-off between model complexity and interpretability. It allows us to create models that are both accurate and transparent. In this example, we'll use the UNOS (United Network for Organ Sharing) heart transplant dataset to illustrate the PRiSM method.

Here, the target outcome we are modelling is "oneyearmort", which refers to:

- 0: a patient who survived at least one year after heart transplant
- 1: a patient who died within one year of heart transplant

We would like to use the available features to model the likelihood of one-year mortality after heart transplantation, and understand how each feature contributes to that likelihood.

[1]:
```
%load_ext autoreload
%autoreload 2

import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt
import seaborn as sns

from prism.config import PROCESSED_DATA_DIR, MODELS_DIR
from prism.preprocessing import normalize, feature_summary,␣
 ↪plot_feature_histograms
from prism.maskedmlp import train_mlp_batched, train_maskedmlp
from prism.partial_responses import partial_responses
from prism.response_plot import plot_partial_responses
from prism.nomogram import nomogram, display_nomograms_side_by_side
from prism.lasso import lasso
from prism.device_tools import get_device, get_num_cpu_workers
from prism.metrics import evaluate_model_performance, compare_model_performance
```

```
2024-09-02 16:34:03.979 | INFO     |
prism.config:<module>:11 - PROJ_ROOT path is:

C:\Users\localuser\PRiSM\prism_github
```

[2]:
```
%reload_ext autoreload
```

1.1 1. Setup and Imports

First, we need to import the necessary libraries and set up our environment.

We set up our computational device (CPU or GPU if available) and set a random seed for reproducibility.

We'll use the 'dirac' method for partial responses, which is one of two methods (the other being 'lebesgue') for calculating partial responses. The Dirac method calculates the effect of each feature individually, which can be computationally efficient but may miss some complex interactions.

We set up our computational device (CPU or GPU if available) and set a random seed for reproducibility.

Two methods, 'lebesgue' and 'dirac', can be used to calculate partial responses. The Lebesgue method uses an average over the predicted surface, which can be more stable for some datasets, but more computationally intensive. If you're working with a bigger dataset, or low compute resources, starting with 'dirac' may be better.

[3]:
```
# Set device to 'cpu', or GPU ('cuda' for NVIDIA, 'mps' for Apple Silicon)
# By default, we select the best available device
device = get_device()
print(f"Using device: {device}")

# Set random seed for reproducibility
random_seed = 257
np.random.seed(random_seed)
torch.manual_seed(random_seed)

# Set method and other parameters
partial_response_method = 'lebesgue'
SAVE_METRICS = True

# TODO: setup model saving.
SAVE_MODELS = False
```

```
Using device: cuda
```

For multithreading compute-intensive tasks, the recommended number of workers for GPU processing is 1, otherwise for CPU stick to the default (number of logical cores - 1). However, depending on your hardware, you may want to experiment.

[4]:
```
if device != 'cpu':
    max_workers = 1
```

```
else:
    max_workers = get_num_cpu_workers()
print(f"max_workers = {max_workers}")
```

```
max_workers = 1
```

1.2 2. Load and Preprocess Data

Next, we load the UNOS heart transplant data and perform necessary preprocessing steps.

[5]:
```
# Load data
data_train = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_train.
 ↪csv'))
data_test = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_test.
 ↪csv'))
data_val = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_val.csv'))

# Drop id column
data_train.drop('trr_id_code', axis=1, inplace=True)
data_test.drop('trr_id_code', axis=1, inplace=True)
data_val.drop('trr_id_code', axis=1, inplace=True)

target_col = 'oneyearmort'

X_train = data_train.drop(target_col, axis=1)
y_train = data_train[target_col]

X_test = data_test.drop(target_col, axis=1)
y_test = data_test[target_col]

X_val = data_val.drop(target_col, axis=1)
y_val = data_val[target_col]

# Normalize the data
X_train_normalized, X_test_normalized = normalize(X_train, X_test)
X_val_normalized = normalize(X_val)

# Convert to tensors
X_train_tensor = torch.tensor(X_train_normalized.values, dtype=torch.float32,␣
 ↪device=device)
X_test_tensor = torch.tensor(X_test_normalized.values, dtype=torch.float32,␣
 ↪device=device)
X_val_tensor = torch.tensor(X_val_normalized.values, dtype=torch.float32,␣
 ↪device=device)

y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32,␣
 ↪device=device)
```

```
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32, device=device)
y_val_tensor = torch.tensor(y_val.values, dtype=torch.float32, device=device)

feature_names = [
    'don age',
    'don isch time min',
    'rec age',
    'rec creat',
    'rec infect 2wk',
    'rec vent',
    'rec sex',
    'tx year',
    'ICM',
    'NICM',
    'prior tx'
]
```

1.3 3. Data Overview

Before building our models, let's get an overview of the data.

[6]:
```
print("Shape of training data:", X_train.shape)
print("Shape of testing data:", X_test.shape)
print("Shape of validation data:", X_val.shape)
print(f"\nClass distribution in training set ({X_train['tx_year'].min()} -␣
  ↪{X_train['tx_year'].max()}):")
print(y_train.value_counts(normalize=True))
print(f"\nClass distribution in test set ({X_test['tx_year'].min()} -␣
  ↪{X_test['tx_year'].max()}):")
print(y_test.value_counts(normalize=True))
print(f"\nClass distribution in validation set ({X_val['tx_year'].min()} -␣
  ↪{X_val['tx_year'].max()}):")
print(y_val.value_counts(normalize=True))
```

```
Shape of training data: (31315, 11)
Shape of testing data: (6120, 11)
Shape of validation data: (4750, 11)

Class distribution in training set (1997 - 2013):
oneyearmort
0    0.877024
1    0.122976
Name: proportion, dtype: float64

Class distribution in test set (2014 - 2016):
oneyearmort
0    0.89281
1    0.10719
```

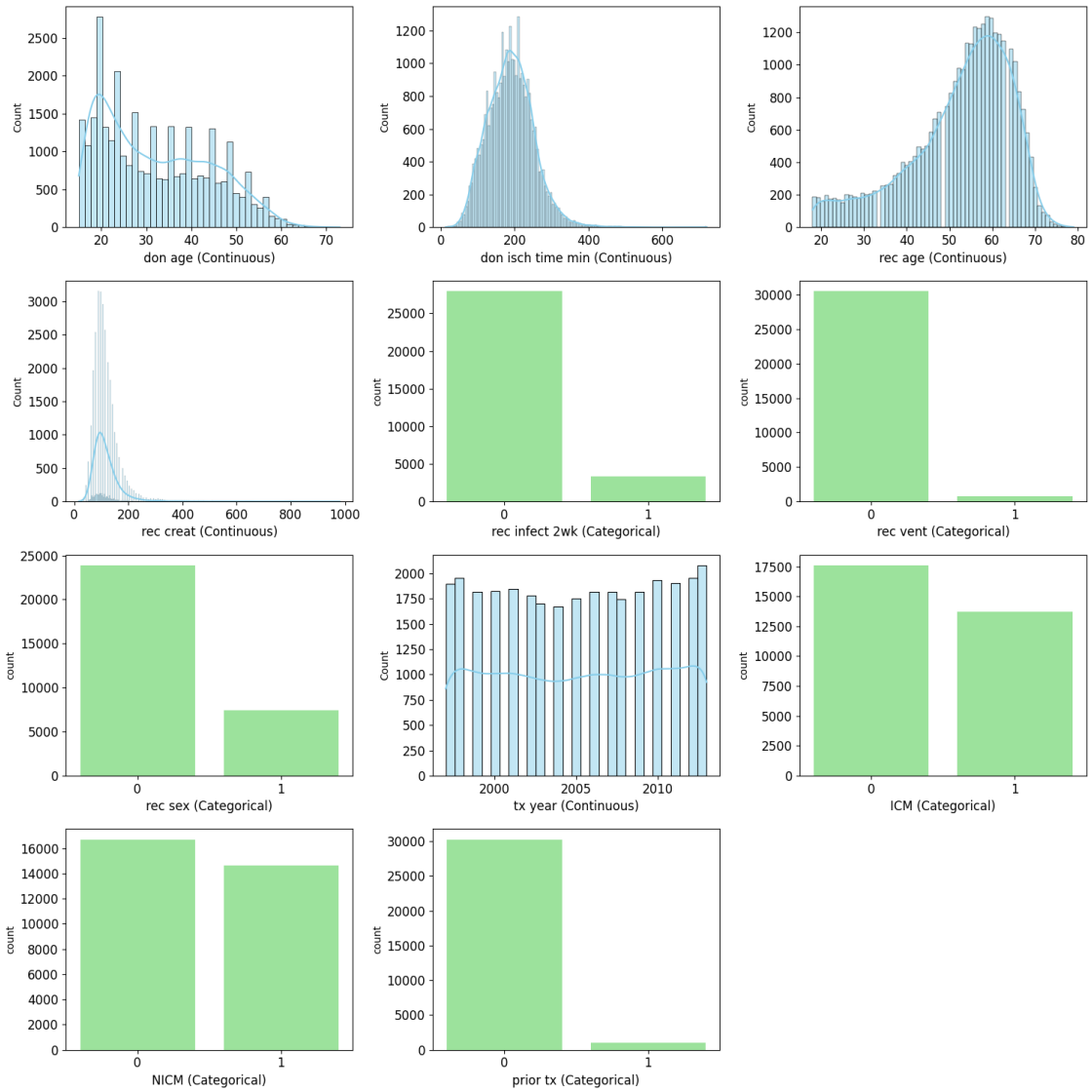
```
Name: proportion, dtype: float64

Class distribution in validation set (2017 - 2018):
oneyearmort
0    0.900842
1    0.099158
Name: proportion, dtype: float64
```

[7]:
```
feature_stats = feature_summary(X_train, categorical_threshold=15)
print("Feature Statistics:")
print(feature_stats)
```

```
Feature Statistics:
                          Data Type  Non-Null Count  Null Count          Mean  \
donage                        int64           31315           0     32.091043
donischemictimeminutes      float64           31315           0    189.745752
recageyear                    int64           31315           0     52.359412
reccreatininemostrecent     float64           31315           0    119.943712
recinfections2weeks           int64           31315           0      0.106754
recventilator                 int64           31315           0      0.025770
recsex                        int64           31315           0      0.237043
tx_year                       int64           31315           0   2005.084943
diagnICM                      int64           31315           0      0.437841
diagnNICM                     int64           31315           0      0.466741
recpriortx                    int64           31315           0      0.034424

                             Median    Std Dev        IQR          Min  \
donage                    30.00000  12.018143  21.000000    15.000000
donischemictimeminutes   187.99805  62.573339  82.998050    12.998657
recageyear                55.00000  12.169868  15.000000    18.000000
reccreatininemostrecent  106.08000  67.657171  44.200008    16.796000
recinfections2weeks        0.00000   0.308805   0.000000     0.000000
recventilator              0.00000   0.158452   0.000000     0.000000
recsex                     0.00000   0.425276   0.000000     0.000000
tx_year                 2005.00000   4.996192   9.000000  1997.000000
diagnICM                   0.00000   0.496129   1.000000     0.000000
diagnNICM                  0.00000   0.498901   1.000000     0.000000
recpriortx                 0.00000   0.182320   0.000000     0.000000

                              Max  Unique Values  Is Categorical
donage                   73.00000             59           False
donischemictimeminutes  720.00000            454           False
recageyear               79.00000             62           False
reccreatininemostrecent 981.23999            396           False
recinfections2weeks       1.00000              2            True
recventilator             1.00000              2            True
recsex                    1.00000              2            True
tx_year                2013.00000             17           False
```

```
diagnICM                    1.00000             2           True
diagnNICM                   1.00000             2           True
recpriortx                  1.00000             2           True
```

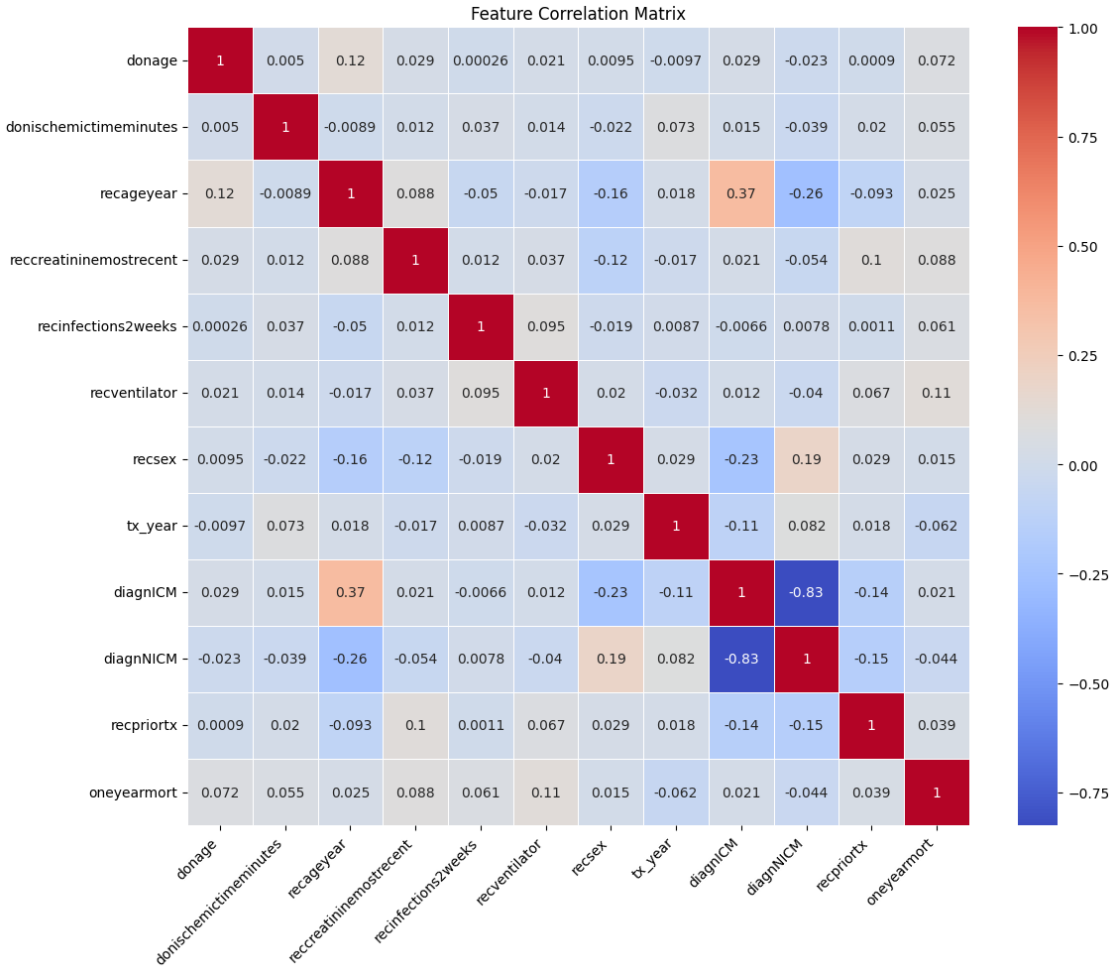
[8]:
```
plot_feature_histograms(X_train, feature_stats, figsize=(15,15),␣
↪feature_names=feature_names)
```



[8]:
```
<module 'matplotlib.pyplot' from
'c:\\Users\\localuser\\PRiSM\\prism_github\\venv_prism\\Lib\\site-
packages\\matplotlib\\pyplot.py'>
```

[9]:
```
# Correlation matrix
correlation_matrix = pd.concat([X_train, y_train], axis=1).corr()
```

```
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Feature Correlation Matrix')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



1.4 4. Train Initial Black Box Model (MLP)

We start by training a black box model, in this case, a Multi-Layer Perceptron (MLP) with 10 nodes in the hidden layer. This model serves as our baseline and represents the type of hard-to-interpret model that is often used in machine learning.

[10]:
```
mlp_hyperparameters = {
    'n_hidden': 10,
    'weight_decay': 1e-5,
    'lr': 0.001,
```

```
    'patience': 50,
    'tolerance': 0.0001,
    'batch_size': 1024,
    'device': device,
    'seed': random_seed
}

blackbox_model = train_mlp_batched(X_train_normalized, y_train,␣
 ↪X_test_normalized, y_test, **mlp_hyperparameters)
```

```
Epoch 0: Train loss 0.6690, Test loss 0.6166
Epoch 1: Train loss 0.5749, Test loss 0.5303
Epoch 2: Train loss 0.5062, Test loss 0.4680
Epoch 3: Train loss 0.4573, Test loss 0.4224
Epoch 4: Train loss 0.4240, Test loss 0.3906
Epoch 5: Train loss 0.4014, Test loss 0.3693
Epoch 6: Train loss 0.3869, Test loss 0.3563
Epoch 7: Train loss 0.3782, Test loss 0.3485
Epoch 8: Train loss 0.3726, Test loss 0.3440
Epoch 9: Train loss 0.3694, Test loss 0.3416
Epoch 10: Train loss 0.3667, Test loss 0.3402
Epoch 11: Train loss 0.3653, Test loss 0.3395
Epoch 12: Train loss 0.3644, Test loss 0.3393
Epoch 13: Train loss 0.3640, Test loss 0.3392
Epoch 14: Train loss 0.3631, Test loss 0.3391
Epoch 15: Train loss 0.3621, Test loss 0.3391
Epoch 16: Train loss 0.3624, Test loss 0.3392
Epoch 17: Train loss 0.3617, Test loss 0.3392
Epoch 18: Train loss 0.3613, Test loss 0.3393
Epoch 19: Train loss 0.3611, Test loss 0.3392
Epoch 20: Train loss 0.3611, Test loss 0.3393
Epoch 21: Train loss 0.3608, Test loss 0.3394
Epoch 22: Train loss 0.3601, Test loss 0.3392
Epoch 23: Train loss 0.3604, Test loss 0.3392
Epoch 24: Train loss 0.3604, Test loss 0.3392
Epoch 25: Train loss 0.3601, Test loss 0.3390
Epoch 26: Train loss 0.3598, Test loss 0.3390
Epoch 27: Train loss 0.3593, Test loss 0.3389
Epoch 28: Train loss 0.3598, Test loss 0.3391
Epoch 29: Train loss 0.3586, Test loss 0.3388
Epoch 30: Train loss 0.3588, Test loss 0.3389
Epoch 31: Train loss 0.3587, Test loss 0.3388
Epoch 32: Train loss 0.3584, Test loss 0.3389
Epoch 33: Train loss 0.3585, Test loss 0.3388
Epoch 34: Train loss 0.3584, Test loss 0.3388
Epoch 35: Train loss 0.3588, Test loss 0.3387
Epoch 36: Train loss 0.3583, Test loss 0.3386
Epoch 37: Train loss 0.3581, Test loss 0.3386
```

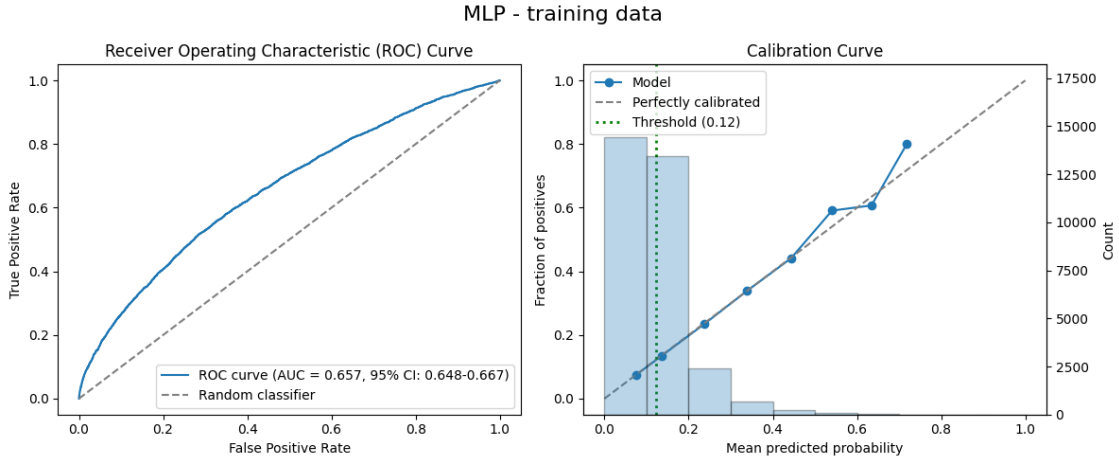
```
Epoch 38: Train loss 0.3583, Test loss 0.3385
Epoch 39: Train loss 0.3576, Test loss 0.3384
Epoch 40: Train loss 0.3577, Test loss 0.3384
Epoch 41: Train loss 0.3574, Test loss 0.3385
Epoch 42: Train loss 0.3572, Test loss 0.3384
Epoch 43: Train loss 0.3567, Test loss 0.3383
Epoch 44: Train loss 0.3573, Test loss 0.3385
Epoch 45: Train loss 0.3568, Test loss 0.3383
Epoch 46: Train loss 0.3569, Test loss 0.3384
Epoch 47: Train loss 0.3573, Test loss 0.3383
Epoch 48: Train loss 0.3570, Test loss 0.3380
Epoch 49: Train loss 0.3573, Test loss 0.3382
Epoch 50: Train loss 0.3567, Test loss 0.3381
Epoch 51: Train loss 0.3571, Test loss 0.3381
Epoch 52: Train loss 0.3568, Test loss 0.3379
Epoch 53: Train loss 0.3562, Test loss 0.3378
Epoch 54: Train loss 0.3567, Test loss 0.3377
Epoch 55: Train loss 0.3564, Test loss 0.3379
Epoch 56: Train loss 0.3566, Test loss 0.3380
Epoch 57: Train loss 0.3558, Test loss 0.3378
Epoch 58: Train loss 0.3564, Test loss 0.3379
Epoch 59: Train loss 0.3559, Test loss 0.3376
Epoch 60: Train loss 0.3550, Test loss 0.3377
Epoch 61: Train loss 0.3555, Test loss 0.3378
Epoch 62: Train loss 0.3561, Test loss 0.3378
Epoch 63: Train loss 0.3547, Test loss 0.3377
Epoch 64: Train loss 0.3551, Test loss 0.3377
Epoch 65: Train loss 0.3556, Test loss 0.3378
Epoch 66: Train loss 0.3553, Test loss 0.3375
Epoch 67: Train loss 0.3555, Test loss 0.3377
Epoch 68: Train loss 0.3552, Test loss 0.3376
Epoch 69: Train loss 0.3547, Test loss 0.3377
Epoch 70: Train loss 0.3555, Test loss 0.3376
Epoch 71: Train loss 0.3549, Test loss 0.3378
Epoch 72: Train loss 0.3546, Test loss 0.3376
Epoch 73: Train loss 0.3546, Test loss 0.3376
Epoch 74: Train loss 0.3550, Test loss 0.3375
Epoch 75: Train loss 0.3541, Test loss 0.3375
Epoch 76: Train loss 0.3547, Test loss 0.3378
Epoch 77: Train loss 0.3543, Test loss 0.3376
Epoch 78: Train loss 0.3545, Test loss 0.3375
Epoch 79: Train loss 0.3542, Test loss 0.3374
Epoch 80: Train loss 0.3552, Test loss 0.3375
Epoch 81: Train loss 0.3544, Test loss 0.3374
Epoch 82: Train loss 0.3538, Test loss 0.3376
Epoch 83: Train loss 0.3546, Test loss 0.3375
Epoch 84: Train loss 0.3540, Test loss 0.3374
Epoch 85: Train loss 0.3552, Test loss 0.3374
```

```
Epoch 86: Train loss 0.3548, Test loss 0.3376
Epoch 87: Train loss 0.3542, Test loss 0.3373
Epoch 88: Train loss 0.3540, Test loss 0.3376
Epoch 89: Train loss 0.3542, Test loss 0.3375
Epoch 90: Train loss 0.3542, Test loss 0.3376
Epoch 91: Train loss 0.3540, Test loss 0.3375
Epoch 92: Train loss 0.3543, Test loss 0.3375
Epoch 93: Train loss 0.3537, Test loss 0.3374
Epoch 94: Train loss 0.3539, Test loss 0.3377
Epoch 95: Train loss 0.3543, Test loss 0.3375
Epoch 96: Train loss 0.3540, Test loss 0.3374
Epoch 97: Train loss 0.3537, Test loss 0.3377
Epoch 98: Train loss 0.3535, Test loss 0.3374
Epoch 99: Train loss 0.3537, Test loss 0.3374
Epoch 100: Train loss 0.3534, Test loss 0.3375
Epoch 101: Train loss 0.3542, Test loss 0.3376
Epoch 102: Train loss 0.3539, Test loss 0.3376
Epoch 103: Train loss 0.3540, Test loss 0.3377
Epoch 104: Train loss 0.3532, Test loss 0.3373
Epoch 105: Train loss 0.3537, Test loss 0.3378
Epoch 106: Train loss 0.3537, Test loss 0.3375
Epoch 107: Train loss 0.3535, Test loss 0.3374
Epoch 108: Train loss 0.3535, Test loss 0.3377
Epoch 109: Train loss 0.3533, Test loss 0.3377
Epoch 110: Train loss 0.3535, Test loss 0.3377
Epoch 111: Train loss 0.3536, Test loss 0.3376
Epoch 112: Train loss 0.3539, Test loss 0.3379
Epoch 113: Train loss 0.3533, Test loss 0.3376
Epoch 114: Train loss 0.3539, Test loss 0.3377
Epoch 115: Train loss 0.3536, Test loss 0.3379
Epoch 116: Train loss 0.3543, Test loss 0.3379
Epoch 117: Train loss 0.3534, Test loss 0.3376
Epoch 118: Train loss 0.3534, Test loss 0.3376
Epoch 119: Train loss 0.3545, Test loss 0.3378
Epoch 120: Train loss 0.3538, Test loss 0.3379
Epoch 121: Train loss 0.3538, Test loss 0.3374
Epoch 122: Train loss 0.3535, Test loss 0.3374
Epoch 123: Train loss 0.3536, Test loss 0.3377
Epoch 124: Train loss 0.3527, Test loss 0.3378
Epoch 125: Train loss 0.3535, Test loss 0.3378
Epoch 126: Train loss 0.3532, Test loss 0.3377
Epoch 127: Train loss 0.3538, Test loss 0.3380
Epoch 128: Train loss 0.3536, Test loss 0.3378
Epoch 129: Train loss 0.3537, Test loss 0.3378
Epoch 130: Train loss 0.3527, Test loss 0.3378
Epoch 131: Train loss 0.3532, Test loss 0.3376
Epoch 132: Train loss 0.3534, Test loss 0.3377
Epoch 133: Train loss 0.3533, Test loss 0.3378
```

```
Epoch 134: Train loss 0.3529, Test loss 0.3377
Epoch 135: Train loss 0.3537, Test loss 0.3379
Epoch 136: Train loss 0.3535, Test loss 0.3380
Stopping early at epoch 87
```

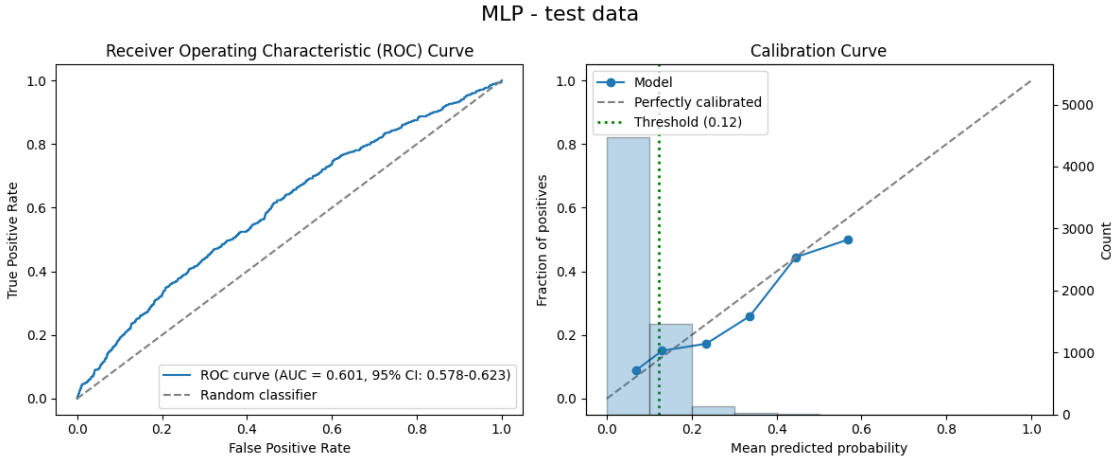
[11]:
```
y_pred_train_blackbox = blackbox_model.predict(X_train_tensor)
y_pred_test_blackbox = blackbox_model.predict(X_test_tensor)

# Evaluate MLP
results_mlp_train = evaluate_model_performance(y_train, y_pred_train_blackbox,␣
 ↪y_train, title="MLP - training data")
results_mlp_test = evaluate_model_performance(y_test, y_pred_test_blackbox,␣
 ↪y_train, title="MLP - test data")
```

```
---- Model Performance Metrics MLP - training data ----
Threshold (prevalence) set to: 0.123
AUROC:          0.657 (95% CI: 0.648-0.667) (Area Under the Receiver Operating
Characteristic curve)
Accuracy:       0.652 (Proportion of correct predictions)
Precision:      0.191 (Proportion of true positives among positive predictions,
aka PPV)
Recall:         0.564 (Proportion of true positives among actual positives, aka
sensitivity)
F1 Score:       0.285 (Harmonic mean of precision and recall)
-----------------------------------------------
```




```
---- Model Performance Metrics MLP - test data ----
Threshold (prevalence) set to: 0.123
AUROC:          0.601 (95% CI: 0.578-0.623) (Area Under the Receiver Operating
Characteristic curve)
```

```
Accuracy:       0.797 (Proportion of correct predictions)
Precision:      0.175 (Proportion of true positives among positive predictions,
aka PPV)
Recall:         0.241 (Proportion of true positives among actual positives, aka
sensitivity)
F1 Score:       0.203 (Harmonic mean of precision and recall)
-----------------------------------------
```



1.5 5. Calculate Partial Responses

Now we calculate partial responses for our black box model. This is a key step in the PRiSM method. Partial responses help us understand how individual features or pairs of features contribute to the model's predictions, making the black box model more interpretable.

[12]:
```
partial_responses_params = {
    'x_train': X_train_tensor,
    'method': partial_response_method,
    'device': device,
    'batch_size': 512,
    'max_workers': max_workers
}

partial_responses_train = partial_responses(X_train_tensor, blackbox_model,␣
 ↪**partial_responses_params)
partial_responses_test = partial_responses(X_test_tensor, blackbox_model,␣
 ↪**partial_responses_params)
```

```
Main compute device: cuda
Max threads: 1
Batch size: 512
Univariate 0,   (cuda)
Univariate 1,   (cuda)
```

```
Univariate 2,   (cuda)
Univariate 3,   (cuda)
Univariate 4,   (cuda)
Univariate 5,   (cuda)
Univariate 6,   (cuda)
Univariate 7,   (cuda)
Univariate 8,   (cuda)
Univariate 9,   (cuda)
Univariate 10,  (cuda)
Bivariate 0,1,  (cuda)
Bivariate 0,2,  (cuda)
Bivariate 0,3,  (cuda)
Bivariate 0,4,  (cuda)
Bivariate 0,5,  (cuda)
Bivariate 0,6,  (cuda)
Bivariate 0,7,  (cuda)
Bivariate 0,8,  (cuda)
Bivariate 0,9,  (cuda)
Bivariate 0,10, (cuda)
Bivariate 1,2,  (cuda)
Bivariate 1,3,  (cuda)
Bivariate 1,4,  (cuda)
Bivariate 1,5,  (cuda)
Bivariate 1,6,  (cuda)
Bivariate 1,7,  (cuda)
Bivariate 1,8,  (cuda)
Bivariate 1,9,  (cuda)
Bivariate 1,10, (cuda)
Bivariate 2,3,  (cuda)
Bivariate 2,4,  (cuda)
Bivariate 2,5,  (cuda)
Bivariate 2,6,  (cuda)
Bivariate 2,7,  (cuda)
Bivariate 2,8,  (cuda)
Bivariate 2,9,  (cuda)
Bivariate 2,10, (cuda)
Bivariate 3,4,  (cuda)
Bivariate 3,5,  (cuda)
Bivariate 3,6,  (cuda)
Bivariate 3,7,  (cuda)
Bivariate 3,8,  (cuda)
Bivariate 3,9,  (cuda)
Bivariate 3,10, (cuda)
Bivariate 4,5,  (cuda)
Bivariate 4,6,  (cuda)
Bivariate 4,7,  (cuda)
Bivariate 4,8,  (cuda)
Bivariate 4,9,  (cuda)
```

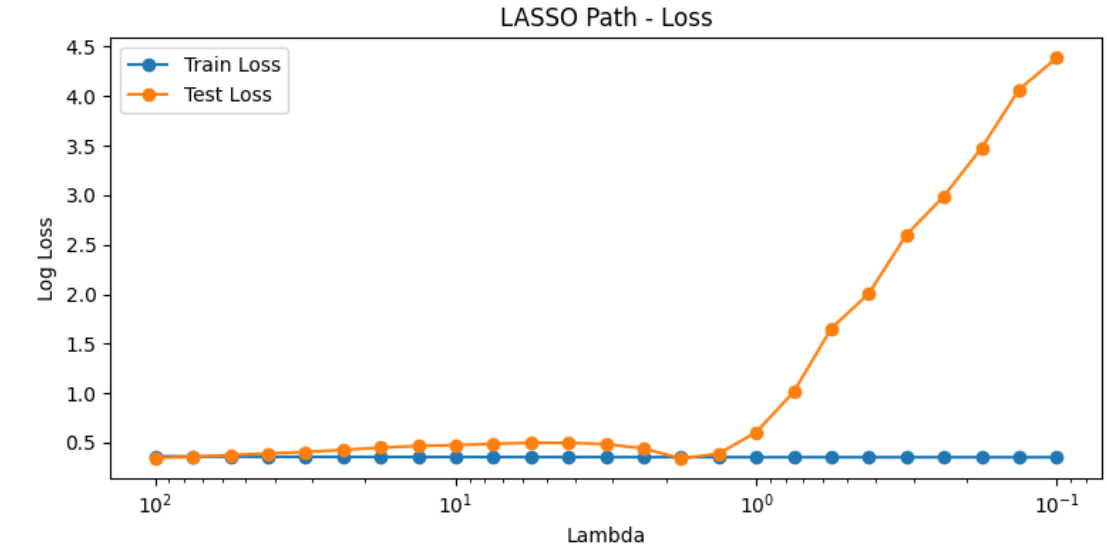
```
Bivariate 4,10, (cuda)
Bivariate 5,6,  (cuda)
Bivariate 5,7,  (cuda)
Bivariate 5,8,  (cuda)
Bivariate 5,9,  (cuda)
Bivariate 5,10, (cuda)
Bivariate 6,7,  (cuda)
Bivariate 6,8,  (cuda)
Bivariate 6,9,  (cuda)
Bivariate 6,10, (cuda)
Bivariate 7,8,  (cuda)
Bivariate 7,9,  (cuda)
Bivariate 7,10, (cuda)
Bivariate 8,9,  (cuda)
Bivariate 8,10, (cuda)
Bivariate 9,10, (cuda)
Main compute device: cuda
Max threads: 1
Batch size: 512
Univariate 0,   (cuda)
Univariate 1,   (cuda)
Univariate 2,   (cuda)
Univariate 3,   (cuda)
Univariate 4,   (cuda)
Univariate 5,   (cuda)
Univariate 6,   (cuda)
Univariate 7,   (cuda)
Univariate 8,   (cuda)
Univariate 9,   (cuda)
Univariate 10,  (cuda)
Bivariate 0,1,  (cuda)
Bivariate 0,2,  (cuda)
Bivariate 0,3,  (cuda)
Bivariate 0,4,  (cuda)
Bivariate 0,5,  (cuda)
Bivariate 0,6,  (cuda)
Bivariate 0,7,  (cuda)
Bivariate 0,8,  (cuda)
Bivariate 0,9,  (cuda)
Bivariate 0,10, (cuda)
Bivariate 1,2,  (cuda)
Bivariate 1,3,  (cuda)
Bivariate 1,4,  (cuda)
Bivariate 1,5,  (cuda)
Bivariate 1,6,  (cuda)
Bivariate 1,7,  (cuda)
Bivariate 1,8,  (cuda)
Bivariate 1,9,  (cuda)
```

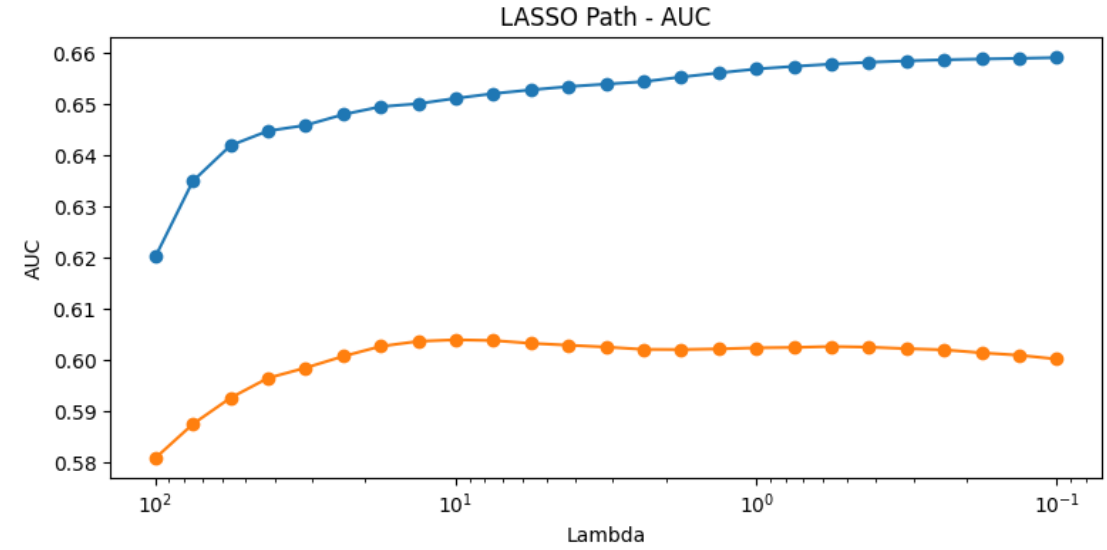
```
Bivariate 1,10, (cuda)
Bivariate 2,3,  (cuda)
Bivariate 2,4,  (cuda)
Bivariate 2,5,  (cuda)
Bivariate 2,6,  (cuda)
Bivariate 2,7,  (cuda)
Bivariate 2,8,  (cuda)
Bivariate 2,9,  (cuda)
Bivariate 2,10, (cuda)
Bivariate 3,4,  (cuda)
Bivariate 3,5,  (cuda)
Bivariate 3,6,  (cuda)
Bivariate 3,7,  (cuda)
Bivariate 3,8,  (cuda)
Bivariate 3,9,  (cuda)
Bivariate 3,10, (cuda)
Bivariate 4,5,  (cuda)
Bivariate 4,6,  (cuda)
Bivariate 4,7,  (cuda)
Bivariate 4,8,  (cuda)
Bivariate 4,9,  (cuda)
Bivariate 4,10, (cuda)
Bivariate 5,6,  (cuda)
Bivariate 5,7,  (cuda)
Bivariate 5,8,  (cuda)
Bivariate 5,9,  (cuda)
Bivariate 5,10, (cuda)
Bivariate 6,7,  (cuda)
Bivariate 6,8,  (cuda)
Bivariate 6,9,  (cuda)
Bivariate 6,10, (cuda)
Bivariate 7,8,  (cuda)
Bivariate 7,9,  (cuda)
Bivariate 7,10, (cuda)
Bivariate 8,9,  (cuda)
Bivariate 8,10, (cuda)
Bivariate 9,10, (cuda)
```

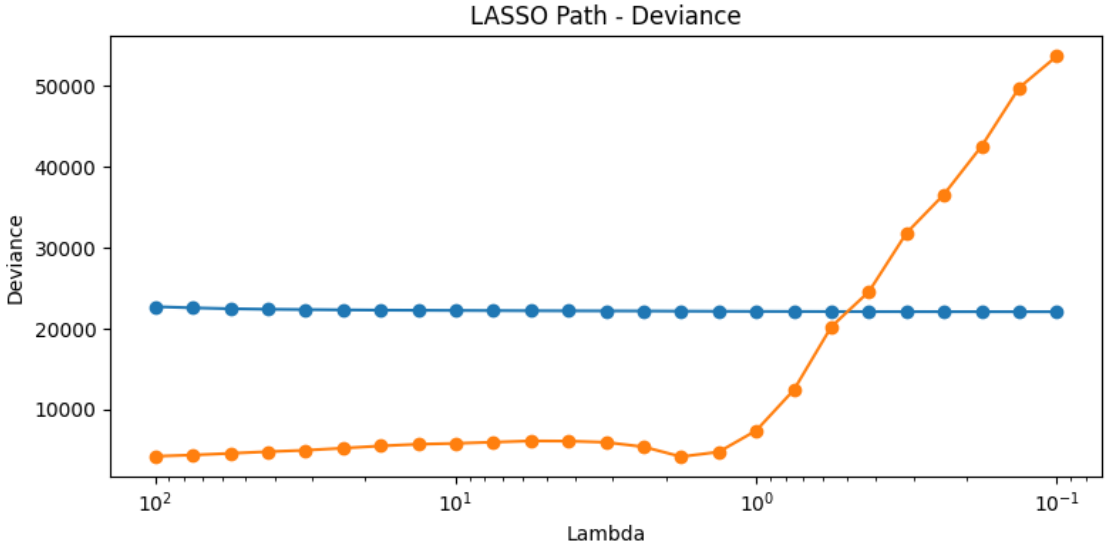
1.6 6. Perform LASSO on Partial Responses

We use LASSO (Least Absolute Shrinkage and Selection Operator) to select the most consequential partial responses. This step helps us identify which features or feature interactions contribute most to the model's predictions. A range of regularization strengths (lambda) are used, and for each one a logistic regression model with LASSO regularization is trained and evaluated. Based on the results, we can choose the most suitable lambda, balancing model performance against the total number of selected features.

[13]:
```
lasso_results = lasso(
    partial_responses_train,
    partial_responses_test,
    y_train,
    y_test,
    feature_names=feature_names,
    nlambda=25,
    min_lambda=0.1,
    max_lambda=100,
    batch_size=2,
    seed=random_seed
)
```

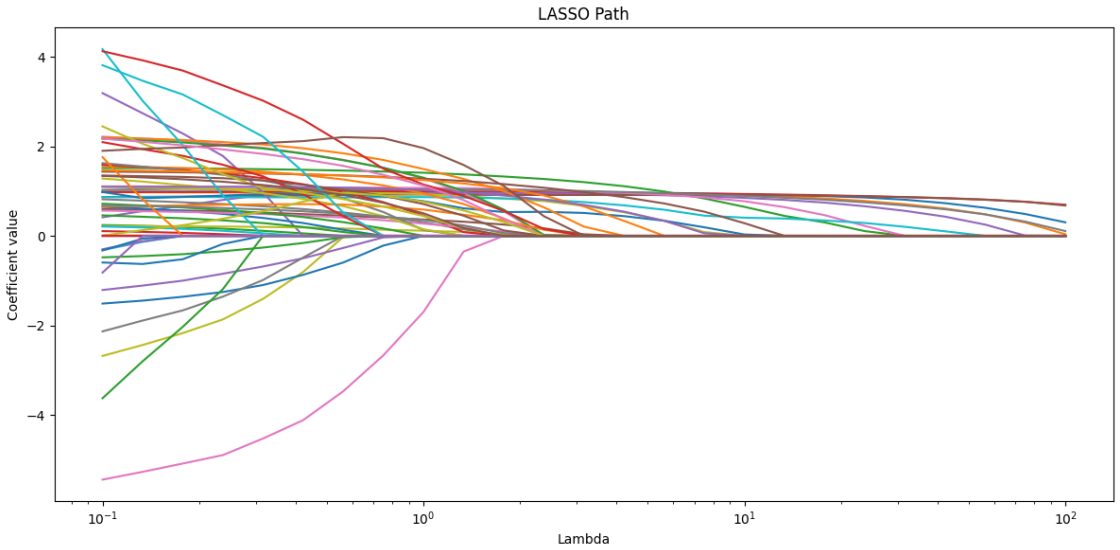

LASSO Path - Loss
Train Loss
Test Loss
Log Loss
Lambda


LASSO Path - AUC
AUC
Lambda


LASSO Path - Deviance
Deviance
Lambda

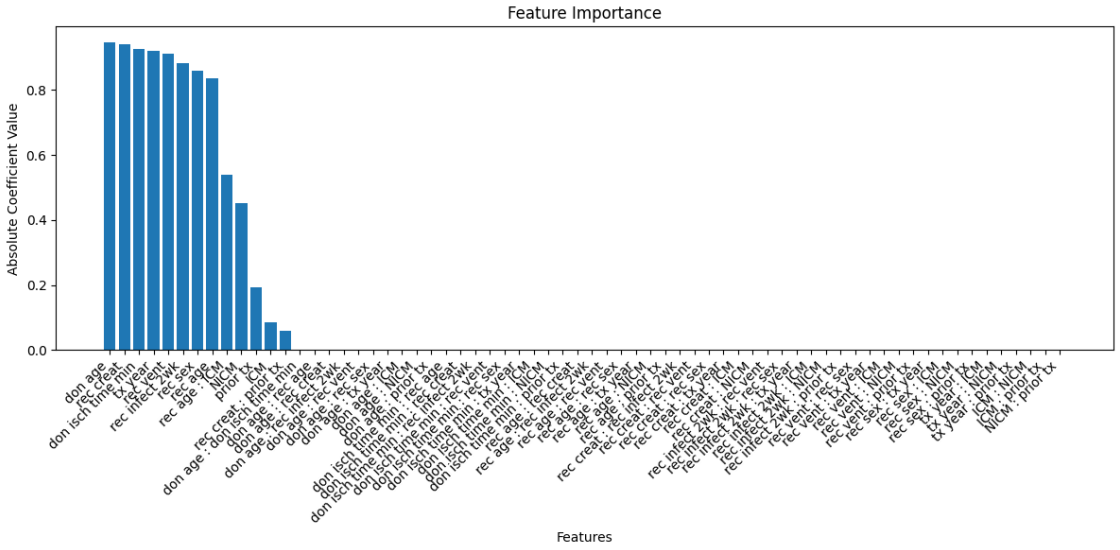
```
<IPython.core.display.HTML object>
All fits converged successfully.
```

[14]:
```
lasso_results.plot_lambda_path()
```



[15]:
```
# lasso_results.select_lambda_max_test_auc()

# Alternatively, manually select lambda as needed
lasso_results.select_lambda(9)
```

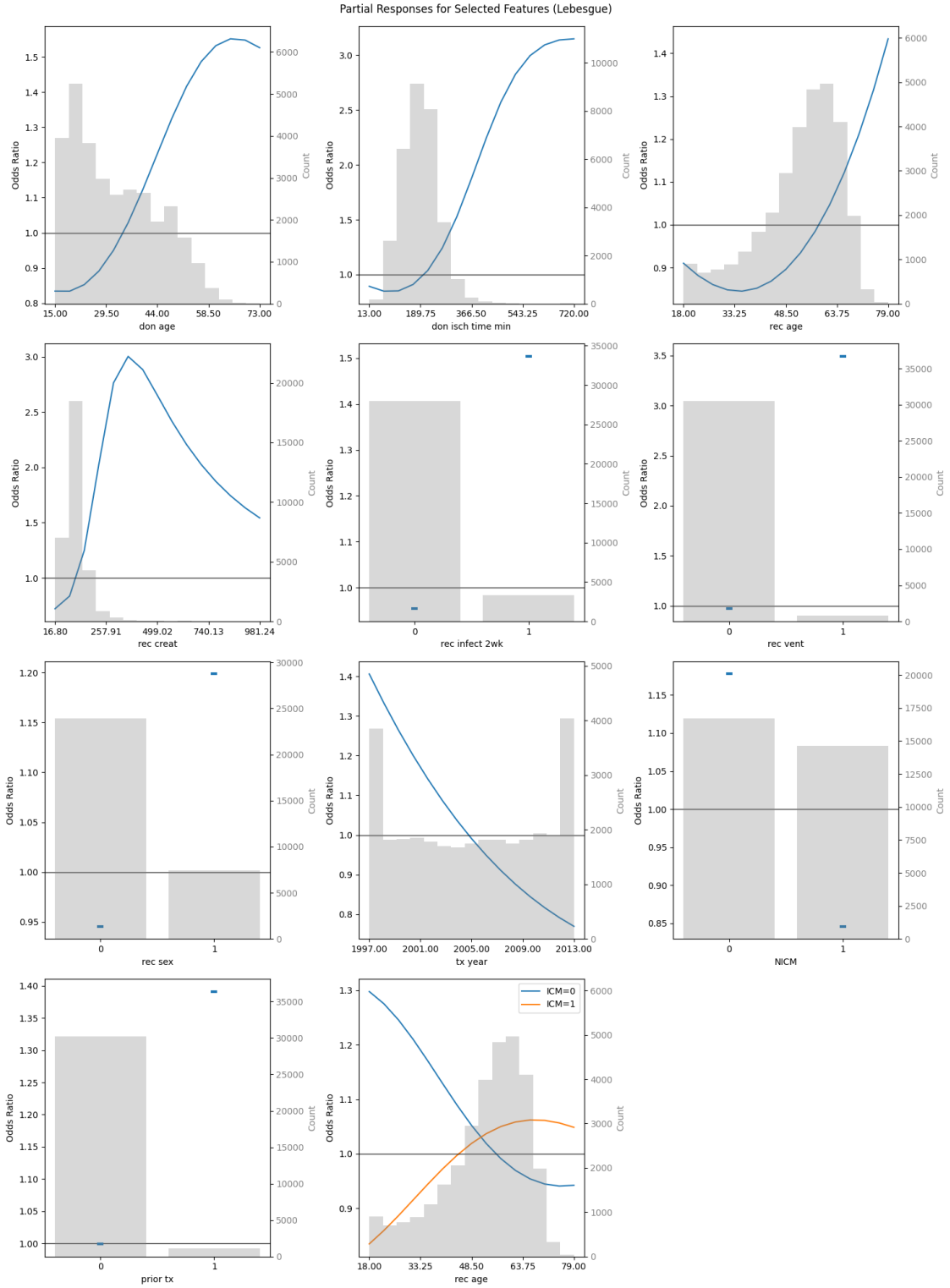
[16]:
```
lasso_results.plot_feature_importance()
```



1.7 7. View Selected Partial Responses and Generate Nomogram

First, we visualize the contribution of each partial response selected with LASSO to the target value.

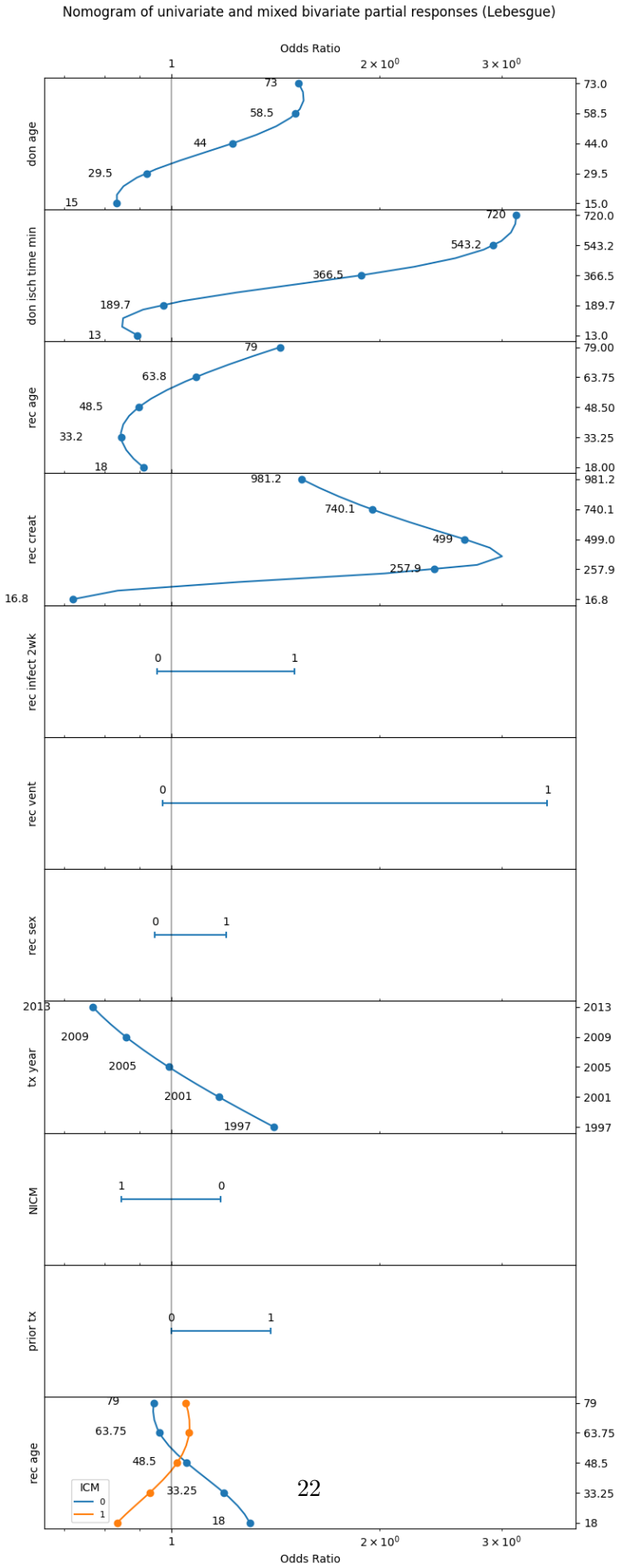
[17]:
```
fig_blackbox = plot_partial_responses(
    lasso_results,
    X_train_tensor,
    X_train.median().values,
    X_train.std().values,
    blackbox_model,
    n_steps=15,
    sd_scale=2,
    method=partial_response_method,
    device=device,
    categorical_threshold=15,
    subtract_univariate=True,
    figsize=(15,20),
    show_fig=True,
    return_fig=True,
    use_odds_ratio=True
)
```



We can also visualize the contribution of each partial response using a nomogram. Nomograms are particularly useful in clinical settings as they provide an intuitive visual representation of a model's

decision-making process.

[18]:
```
nomogram_params = {
    'n_steps': 15,
    'sd_scale': 2,
    'method': partial_response_method,
    'device': device,
    'categorical_threshold': 15,
    'subtract_univariate': True,
    'return_fig': True,
    'use_odds_ratio': True
}

nomogram_results = nomogram(
    lasso_results,
    X_train_tensor,
    X_train.median().values,
    X_train.std().values,
    blackbox_model,
    **nomogram_params
)
nomogram_main_mlp = nomogram_results[6]
nomogram_non_mixed_mlp = nomogram_results[7]
```


Nomogram of univariate and mixed bivariate partial responses (Lebesgue)
Odds Ratio
don age
don isch time min
rec age
rec creat
rec infect 2wk
rec vent
rec sex
tx year
NICM
prior tx
rec age
ICM
0
1
Odds Ratio

1.8 8. Train Partial Response Network (PRN)

Now we train a Partial Response Network based on the selected features. The PRN is a type of neural network that incorporates the interpretability of partial responses while maintaining the predictive power of the original black box model.

From the results of the lasso feature selection, we create an input mask for the PRN. The mask is a binary matrix that determines which input features connect to which hidden nodes (subnets) in the network. This enforces the network structure based on selected features.

The PRN uses a subnet architecture, where groups of hidden nodes are dedicated to specific features or feature interactions:

- Univariate subnets: Handle individual feature effects
- Bivariate subnets: Capture interactions between two features
- The number of nodes per subnet is controlled by the subnet_nodes parameter in `mlpmask_pytorch` (default 5)

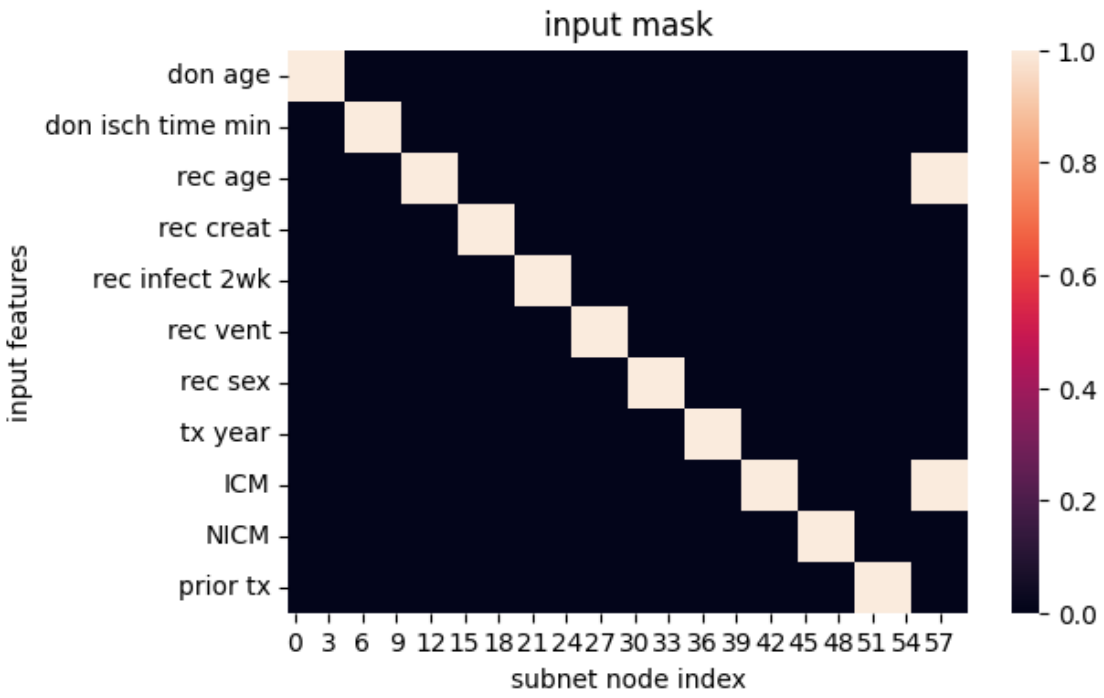
This gives us a resulting network structure as follows:

- Input layer: Original features
- Hidden layer: Structured into subnets based on the mask
- Output layer: Single node for binary classification

The resulting PRN maintains interpretability because each subnet corresponds to a specific feature or interaction. Therefore, the contribution of each feature/interaction can be analyzed separately, and the overall prediction is a sum of these interpretable components.

[19]:
```
mask, n_features = lasso_results.get_mask()
```

```
Selected features: ['don age', 'don isch time min', 'rec age', 'rec creat', 'rec
infect 2wk', 'rec vent', 'rec sex', 'tx year', 'NICM', 'prior tx', 'rec age :
ICM']
```



[20]:
```
prn_hyperparameters = {
    'n_hidden': n_features,
    'mask': mask,
    'subnet_nodes': 5,
    'iter': 10000,
    'lr': 0.05,
    'weight_decay': 0.00001,
    'tolerance': 0.0001,
    'patience': 100,
    'device': device,
    'seed': random_seed
}

partial_response_network = train_maskedmlp(X_train_normalized, y_train,␣
 ↪X_test_normalized, y_test, **prn_hyperparameters)
```

```
Epoch 0, Training loss 0.6397774815559387, Test loss 0.49593019485473633
Epoch 1, Training loss 0.526766836643219, Test loss 0.37622636556625366
Epoch 2, Training loss 0.4034844636917114, Test loss 0.3410705327987671
Epoch 3, Training loss 0.3746626079082489, Test loss 0.3626147508621216
Epoch 4, Training loss 0.4021831452846527, Test loss 0.3887971341609955
Epoch 5, Training loss 0.42837005853652954, Test loss 0.39575478434562683
Epoch 6, Training loss 0.42989999055862427, Test loss 0.3839559555053711
Epoch 7, Training loss 0.41042307019233704, Test loss 0.3624988794326782
Epoch 8, Training loss 0.38338083028793335, Test loss 0.3434027433395386
```

```
Epoch 9, Training loss 0.3658364713191986, Test loss 0.3374348282814026
Epoch 10, Training loss 0.3712558448314667, Test loss 0.3432115316390991
Epoch 11, Training loss 0.3910899758338928, Test loss 0.3451085090637207
Epoch 12, Training loss 0.39637643098831177, Test loss 0.33946576714515686
Epoch 13, Training loss 0.38176169991493225, Test loss 0.3363953232765198
Epoch 14, Training loss 0.3651541471481323, Test loss 0.3419114947319031
Epoch 15, Training loss 0.359774112701416, Test loss 0.3527858853340149
Epoch 16, Training loss 0.3647230863571167, Test loss 0.3622486889362335
Epoch 17, Training loss 0.3720245063304901, Test loss 0.36564430594444275
Epoch 18, Training loss 0.3751950263977051, Test loss 0.3620724380016327
Epoch 19, Training loss 0.3723587393760681, Test loss 0.353683739900589
Epoch 20, Training loss 0.3656158149242401, Test loss 0.3443327844142914
Epoch 21, Training loss 0.35917913913726807, Test loss 0.3378555178642273
Epoch 22, Training loss 0.3569243848323822, Test loss 0.3359212875366211
Epoch 23, Training loss 0.3596181869506836, Test loss 0.33678656816482544
Epoch 24, Training loss 0.36380356550216675, Test loss 0.3372383117675781
Epoch 25, Training loss 0.3649052381515503, Test loss 0.3362584710121155
Epoch 26, Training loss 0.36196455359458923, Test loss 0.335559219121933
Epoch 27, Training loss 0.3579656779766083, Test loss 0.33687713742256165
Epoch 28, Training loss 0.35608339309692383, Test loss 0.34002262353897095
Epoch 29, Training loss 0.3569384515285492, Test loss 0.34330081939697266
Epoch 30, Training loss 0.3589467406272888, Test loss 0.34498095512390137
Epoch 31, Training loss 0.36016491055488586, Test loss 0.3443373441696167
Epoch 32, Training loss 0.35969045758247375, Test loss 0.3418477773666382
Epoch 33, Training loss 0.35797983407974243, Test loss 0.3387589156627655
Epoch 34, Training loss 0.3563093841075897, Test loss 0.3363274931907654
Epoch 35, Training loss 0.35580551624298096, Test loss 0.3351100981235504
Epoch 36, Training loss 0.35658860206604004, Test loss 0.33479052782058716
Epoch 37, Training loss 0.3576945960521698, Test loss 0.334759920835495
Epoch 38, Training loss 0.3579944372177124, Test loss 0.33486154675483704
Epoch 39, Training loss 0.3572573959827423, Test loss 0.33544647693634033
Epoch 40, Training loss 0.35622212290763855, Test loss 0.33678120374679565
Epoch 41, Training loss 0.35573455691337585, Test loss 0.33859074115753174
Epoch 42, Training loss 0.35599571466445923, Test loss 0.3401793837547302
Epoch 43, Training loss 0.35655978322029114, Test loss 0.34089523553848267
Epoch 44, Training loss 0.35684603452682495, Test loss 0.3405120372772217
Epoch 45, Training loss 0.35661280155181885, Test loss 0.3393031358718872
Epoch 46, Training loss 0.35607343912124634, Test loss 0.33783096075057983
Epoch 47, Training loss 0.3556581437587738, Test loss 0.33661308884620667
Epoch 48, Training loss 0.3556409478187561, Test loss 0.3358832895755768
Epoch 49, Training loss 0.3559226095676422, Test loss 0.33560240268707275
Epoch 50, Training loss 0.3561555743217468, Test loss 0.33566194772720337
Epoch 51, Training loss 0.35609158873558044, Test loss 0.33603736758232117
Epoch 52, Training loss 0.3557993173599243, Test loss 0.3367341160774231
Epoch 53, Training loss 0.3555470108985901, Test loss 0.3376386761665344
Epoch 54, Training loss 0.35551801323890686, Test loss 0.3384799361228943
Epoch 55, Training loss 0.3556641638278961, Test loss 0.3389545977115631
Epoch 56, Training loss 0.35579657554626465, Test loss 0.3389008343219757
```

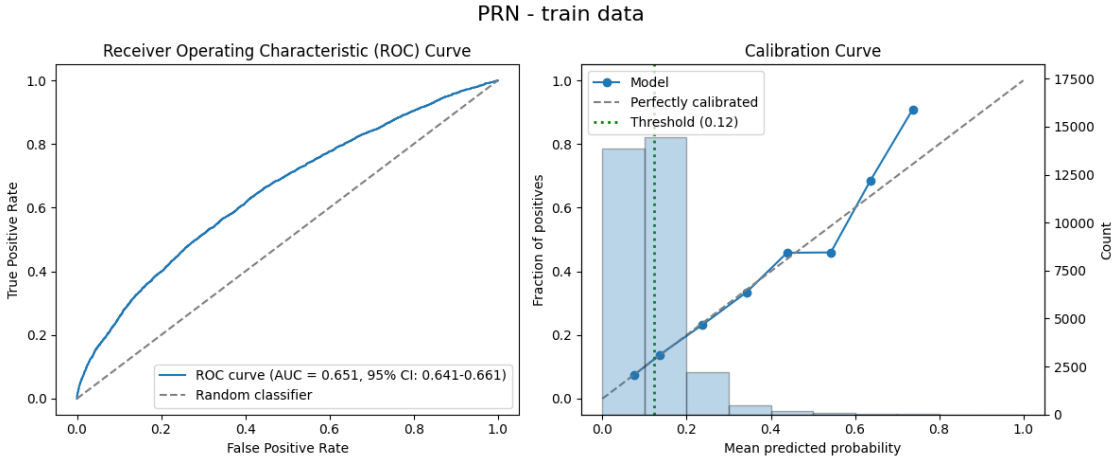
```
Epoch 57, Training loss 0.3557770848274231, Test loss 0.3383820354938507
Epoch 58, Training loss 0.3556273579597473, Test loss 0.33763253688812256
Epoch 59, Training loss 0.3554828464984894, Test loss 0.33691757917404175
Epoch 60, Training loss 0.35545504093170166, Test loss 0.3364109992980957
Epoch 61, Training loss 0.3555319905281067, Test loss 0.33616700768470764
Epoch 62, Training loss 0.35560503602027893, Test loss 0.33617493510246277
Epoch 63, Training loss 0.3555866777896881, Test loss 0.33641141653060913
Epoch 64, Training loss 0.35549312829971313, Test loss 0.33683180809020996
Epoch 65, Training loss 0.35541096329689026, Test loss 0.3373335897922516
Epoch 66, Training loss 0.3554001450538635, Test loss 0.3377601206302643
Epoch 67, Training loss 0.35544121265411377, Test loss 0.33796459436416626
Epoch 68, Training loss 0.35546913743019104, Test loss 0.33788514137268066
Epoch 69, Training loss 0.3554439842700958, Test loss 0.33757326006889343
Epoch 70, Training loss 0.3553844690322876, Test loss 0.3371581435203552
Epoch 71, Training loss 0.3553403913974762, Test loss 0.3367781937122345
Epoch 72, Training loss 0.3553389608860016, Test loss 0.33652815222740173
Epoch 73, Training loss 0.35536089539527893, Test loss 0.33644798398017883
Epoch 74, Training loss 0.35536691546440125, Test loss 0.33653712272644043
Epoch 75, Training loss 0.35534098744392395, Test loss 0.33676472306251526
Epoch 76, Training loss 0.35530340671539307, Test loss 0.3370667099952698
Epoch 77, Training loss 0.35528308153152466, Test loss 0.33735114336013794
Epoch 78, Training loss 0.3552861213684082, Test loss 0.33752575516700745
Epoch 79, Training loss 0.3552936017513275, Test loss 0.33753758668899536
Epoch 80, Training loss 0.3552854359149933, Test loss 0.33739665150642395
Epoch 81, Training loss 0.3552614748477936, Test loss 0.33716654777526855
Epoch 82, Training loss 0.3552381098270416, Test loss 0.3369314968585968
Epoch 83, Training loss 0.3552284240722656, Test loss 0.3367618918418884
Epoch 84, Training loss 0.35522881150245667, Test loss 0.3366968631744385
Epoch 85, Training loss 0.35522526502609253, Test loss 0.33674293756484985
Epoch 86, Training loss 0.3552106022834778, Test loss 0.3368782103061676
Epoch 87, Training loss 0.355191171169281, Test loss 0.3370569348335266
Epoch 88, Training loss 0.35517752170562744, Test loss 0.3372194468975067
Epoch 89, Training loss 0.35517197847366333, Test loss 0.33731162548065186
Epoch 90, Training loss 0.3551677465438843, Test loss 0.33730608224868774
Epoch 91, Training loss 0.3551577627658844, Test loss 0.3372131586074829
Epoch 92, Training loss 0.3551427125930786, Test loss 0.337073415517807
Epoch 93, Training loss 0.35512903332710266, Test loss 0.3369383215904236
Epoch 94, Training loss 0.35512053966522217, Test loss 0.3368504047393799
Epoch 95, Training loss 0.35511451959609985, Test loss 0.3368322253227234
Epoch 96, Training loss 0.3551061153411865, Test loss 0.33688318729400635
Epoch 97, Training loss 0.3550940752029419, Test loss 0.3369826078414917
Epoch 98, Training loss 0.3550817370414734, Test loss 0.33709535002708435
Epoch 99, Training loss 0.35507237911224365, Test loss 0.33718305826187134
Epoch 100, Training loss 0.35506534576416016, Test loss 0.33721789717674255
Epoch 101, Training loss 0.3550574481487274, Test loss 0.33719339966773987
Epoch 102, Training loss 0.3550473749637604, Test loss 0.3371255695819855
Epoch 103, Training loss 0.3550366759300232, Test loss 0.3370445966720581
Epoch 104, Training loss 0.35502758622169495, Test loss 0.33698177337646484
```

```
Epoch 105, Training loss 0.3550202548503876, Test loss 0.33695846796035767
Epoch 106, Training loss 0.3550127446651459, Test loss 0.33698081970214844
Epoch 107, Training loss 0.3550039827823639, Test loss 0.33703920245170593
Epoch 108, Training loss 0.3549947440624237, Test loss 0.3371121883392334
Epoch 109, Training loss 0.35498639941215515, Test loss 0.3371742367744446
Epoch 110, Training loss 0.3549792766571045, Test loss 0.3372052311897278
Epoch 111, Training loss 0.3549722731113434, Test loss 0.33719778060913086
Epoch 112, Training loss 0.35496455430984497, Test loss 0.33715981245040894
Epoch 113, Training loss 0.35495656728744507, Test loss 0.33710983395576477
Epoch 114, Training loss 0.35494914650917053, Test loss 0.3370687663555145
Epoch 115, Training loss 0.35494258999824524, Test loss 0.33705201745033264
Epoch 116, Training loss 0.3549361526966095, Test loss 0.3370647430419922
Epoch 117, Training loss 0.3549293577671051, Test loss 0.3371010422706604
Epoch 118, Training loss 0.35492244362831116, Test loss 0.33714693784713745
Epoch 119, Training loss 0.3549160063266754, Test loss 0.33718565106391907
Epoch 120, Training loss 0.35491010546684265, Test loss 0.3372047245502472
Epoch 121, Training loss 0.35490429401397705, Test loss 0.3372001349925995
Epoch 122, Training loss 0.3548983037471771, Test loss 0.33717775344848633
Epoch 123, Training loss 0.3548923134803772, Test loss 0.3371500074863434
Epoch 124, Training loss 0.3548866808414459, Test loss 0.3371300995349884
Epoch 125, Training loss 0.3548814058303833, Test loss 0.3371271789073944
Epoch 126, Training loss 0.35487616062164307, Test loss 0.33714309334754944
Epoch 127, Training loss 0.35487085580825806, Test loss 0.337172269821167
Epoch 128, Training loss 0.3548656105995178, Test loss 0.3372044861316681
Epoch 129, Training loss 0.3548606336116791, Test loss 0.3372289836406708
Epoch 130, Training loss 0.35485586524009705, Test loss 0.33723902702331543
Epoch 131, Training loss 0.3548511564731598, Test loss 0.33723416924476624
Epoch 132, Training loss 0.35484638810157776, Test loss 0.33722013235092163
Epoch 133, Training loss 0.3548417389392853, Test loss 0.33720576763153076
Epoch 134, Training loss 0.35483723878860474, Test loss 0.3371991515159607
Epoch 135, Training loss 0.3548329174518585, Test loss 0.3372044861316681
Epoch 136, Training loss 0.3548285663127899, Test loss 0.3372207283973694
Early stopping! Epoch 136
Best model at epoch 36
```

[21]:
```
# Evaluate PRN on training data
y_pred_train_prn = partial_response_network.predict(X_train_tensor,␣
 ↪device=device)

results_prn_train = evaluate_model_performance(y_train, y_pred_train_prn,␣
 ↪y_train, title="PRN - train data")

# Evaluate PRN on test data and compare to MLP (blackbox)
y_pred_test_prn = partial_response_network.predict(X_test_tensor, device=device)
```

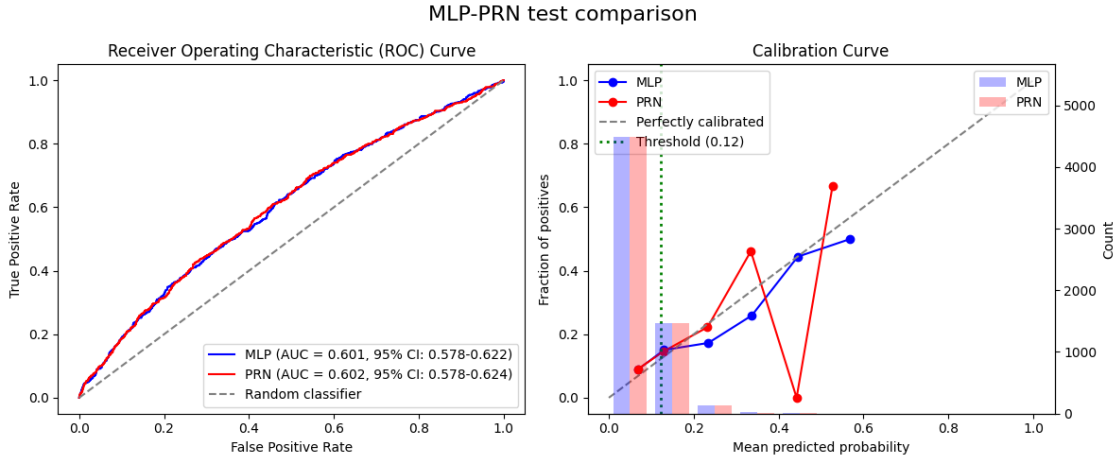
```
results_prn_mlp_test = compare_model_performance(y_test, y_pred_test_blackbox,␣
↪y_pred_test_prn, y_train=y_train, model_names=("MLP","PRN"), title="MLP-PRN␣
↪test comparison")
```

```
---- Model Performance Metrics PRN - train data ----
Threshold (prevalence) set to: 0.123
AUROC:          0.651 (95% CI: 0.641-0.661) (Area Under the Receiver Operating
Characteristic curve)
Accuracy:       0.646 (Proportion of correct predictions)
Precision:      0.187 (Proportion of true positives among positive predictions,
aka PPV)
Recall:         0.561 (Proportion of true positives among actual positives, aka
sensitivity)
F1 Score:       0.280 (Harmonic mean of precision and recall)
-----------------------------------------------
```




```
-------- Model Performance Comparison --------
Threshold (prevalence) set to: 0.123

MLP:
AUROC:          0.601 (95% CI: 0.578-0.622)
Accuracy:       0.797
Precision:      0.175
Recall:         0.241
F1 Score:       0.203

PRN:
AUROC:          0.602 (95% CI: 0.578-0.624)
Accuracy:       0.803
Precision:      0.174
```

```
Recall:          0.223
F1 Score:        0.195
-----------------------------------------------
```



1.9 9. LASSO on PRN

We perform LASSO again, this time on the Partial Response Network to isolate the most important features and interactions.

[22]:
```
partial_responses_params_prn = {
    'x_train': X_train_tensor,
    'method': partial_response_method,
    'device': device,
    'batch_size': 256,
    'max_workers': max_workers
}

partial_responses_train_prn = partial_responses(X_train_tensor,␣
 ↪partial_response_network, **partial_responses_params_prn)
partial_responses_test_prn = partial_responses(X_test_tensor,␣
 ↪partial_response_network, **partial_responses_params_prn)
```

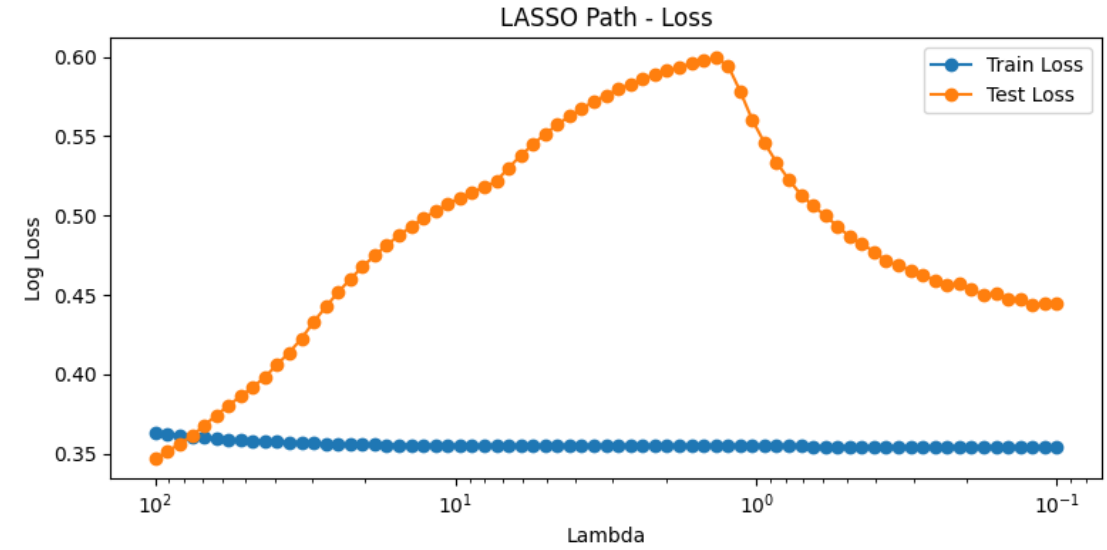
```
Main compute device: cuda
Max threads: 1
Batch size: 256
Univariate 0,   (cuda)
Univariate 1,   (cuda)
Univariate 2,   (cuda)
Univariate 3,   (cuda)
Univariate 4,   (cuda)
Univariate 5,   (cuda)
Univariate 6,   (cuda)
```

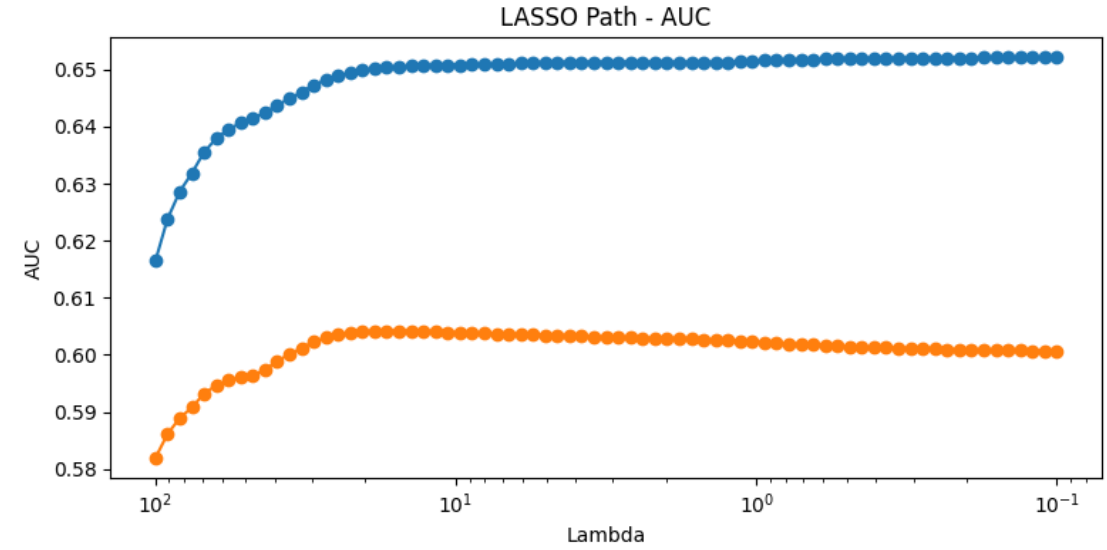
```
Univariate 7,   (cuda)
Univariate 8,   (cuda)
Univariate 9,   (cuda)
Univariate 10,  (cuda)
Bivariate 0,1,  (cuda)
Bivariate 0,2,  (cuda)
Bivariate 0,3,  (cuda)
Bivariate 0,4,  (cuda)
Bivariate 0,5,  (cuda)
Bivariate 0,6,  (cuda)
Bivariate 0,7,  (cuda)
Bivariate 0,8,  (cuda)
Bivariate 0,9,  (cuda)
Bivariate 0,10, (cuda)
Bivariate 1,2,  (cuda)
Bivariate 1,3,  (cuda)
Bivariate 1,4,  (cuda)
Bivariate 1,5,  (cuda)
Bivariate 1,6,  (cuda)
Bivariate 1,7,  (cuda)
Bivariate 1,8,  (cuda)
Bivariate 1,9,  (cuda)
Bivariate 1,10, (cuda)
Bivariate 2,3,  (cuda)
Bivariate 2,4,  (cuda)
Bivariate 2,5,  (cuda)
Bivariate 2,6,  (cuda)
Bivariate 2,7,  (cuda)
Bivariate 2,8,  (cuda)
Bivariate 2,9,  (cuda)
Bivariate 2,10, (cuda)
Bivariate 3,4,  (cuda)
Bivariate 3,5,  (cuda)
Bivariate 3,6,  (cuda)
Bivariate 3,7,  (cuda)
Bivariate 3,8,  (cuda)
Bivariate 3,9,  (cuda)
Bivariate 3,10, (cuda)
Bivariate 4,5,  (cuda)
Bivariate 4,6,  (cuda)
Bivariate 4,7,  (cuda)
Bivariate 4,8,  (cuda)
Bivariate 4,9,  (cuda)
Bivariate 4,10, (cuda)
Bivariate 5,6,  (cuda)
Bivariate 5,7,  (cuda)
Bivariate 5,8,  (cuda)
Bivariate 5,9,  (cuda)
```

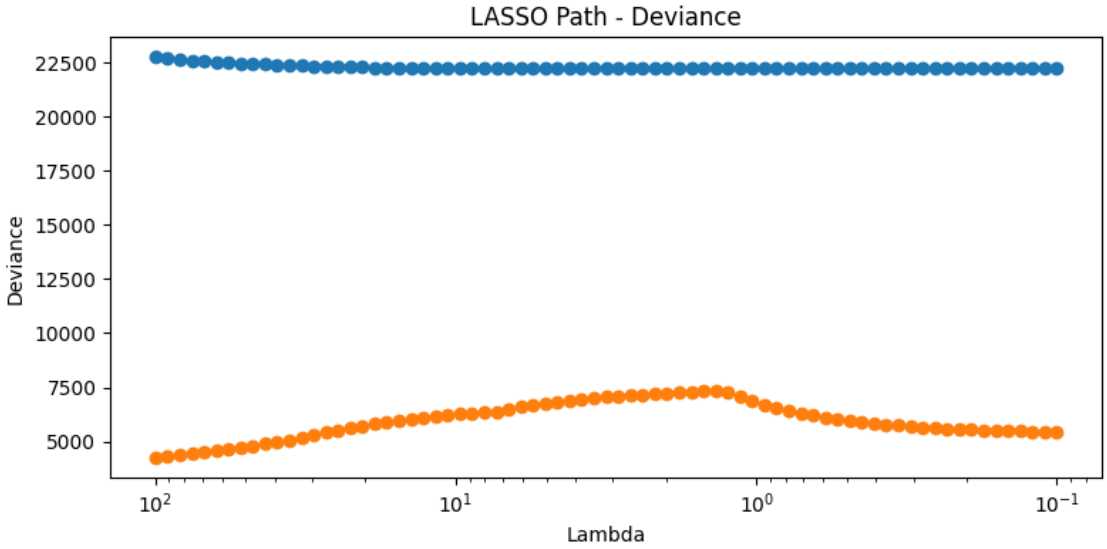
```
Bivariate 5,10, (cuda)
Bivariate 6,7,  (cuda)
Bivariate 6,8,  (cuda)
Bivariate 6,9,  (cuda)
Bivariate 6,10, (cuda)
Bivariate 7,8,  (cuda)
Bivariate 7,9,  (cuda)
Bivariate 7,10, (cuda)
Bivariate 8,9,  (cuda)
Bivariate 8,10, (cuda)
Bivariate 9,10, (cuda)
Main compute device: cuda
Max threads: 1
Batch size: 256
Univariate 0,   (cuda)
Univariate 1,   (cuda)
Univariate 2,   (cuda)
Univariate 3,   (cuda)
Univariate 4,   (cuda)
Univariate 5,   (cuda)
Univariate 6,   (cuda)
Univariate 7,   (cuda)
Univariate 8,   (cuda)
Univariate 9,   (cuda)
Univariate 10,  (cuda)
Bivariate 0,1,  (cuda)
Bivariate 0,2,  (cuda)
Bivariate 0,3,  (cuda)
Bivariate 0,4,  (cuda)
Bivariate 0,5,  (cuda)
Bivariate 0,6,  (cuda)
Bivariate 0,7,  (cuda)
Bivariate 0,8,  (cuda)
Bivariate 0,9,  (cuda)
Bivariate 0,10, (cuda)
Bivariate 1,2,  (cuda)
Bivariate 1,3,  (cuda)
Bivariate 1,4,  (cuda)
Bivariate 1,5,  (cuda)
Bivariate 1,6,  (cuda)
Bivariate 1,7,  (cuda)
Bivariate 1,8,  (cuda)
Bivariate 1,9,  (cuda)
Bivariate 1,10, (cuda)
Bivariate 2,3,  (cuda)
Bivariate 2,4,  (cuda)
Bivariate 2,5,  (cuda)
Bivariate 2,6,  (cuda)
```

```
Bivariate 2,7,  (cuda)
Bivariate 2,8,  (cuda)
Bivariate 2,9,  (cuda)
Bivariate 2,10, (cuda)
Bivariate 3,4,  (cuda)
Bivariate 3,5,  (cuda)
Bivariate 3,6,  (cuda)
Bivariate 3,7,  (cuda)
Bivariate 3,8,  (cuda)
Bivariate 3,9,  (cuda)
Bivariate 3,10, (cuda)
Bivariate 4,5,  (cuda)
Bivariate 4,6,  (cuda)
Bivariate 4,7,  (cuda)
Bivariate 4,8,  (cuda)
Bivariate 4,9,  (cuda)
Bivariate 4,10, (cuda)
Bivariate 5,6,  (cuda)
Bivariate 5,7,  (cuda)
Bivariate 5,8,  (cuda)
Bivariate 5,9,  (cuda)
Bivariate 5,10, (cuda)
Bivariate 6,7,  (cuda)
Bivariate 6,8,  (cuda)
Bivariate 6,9,  (cuda)
Bivariate 6,10, (cuda)
Bivariate 7,8,  (cuda)
Bivariate 7,9,  (cuda)
Bivariate 7,10, (cuda)
Bivariate 8,9,  (cuda)
Bivariate 8,10, (cuda)
Bivariate 9,10, (cuda)
```

[23]:
```
lasso_results_prn = lasso(
    partial_responses_train_prn,
    partial_responses_test_prn,
    y_train,
    y_test,
    feature_names=feature_names,
    nlambda=75,
    min_lambda=0.1,
    max_lambda=100,
    batch_size=2,
    seed=random_seed
)
```


LASSO Path - Loss
Train Loss
Test Loss
Log Loss
0.60
0.55
0.50
0.45
0.40
0.35
10^2
10^1
10^0
10^{-1}
Lambda

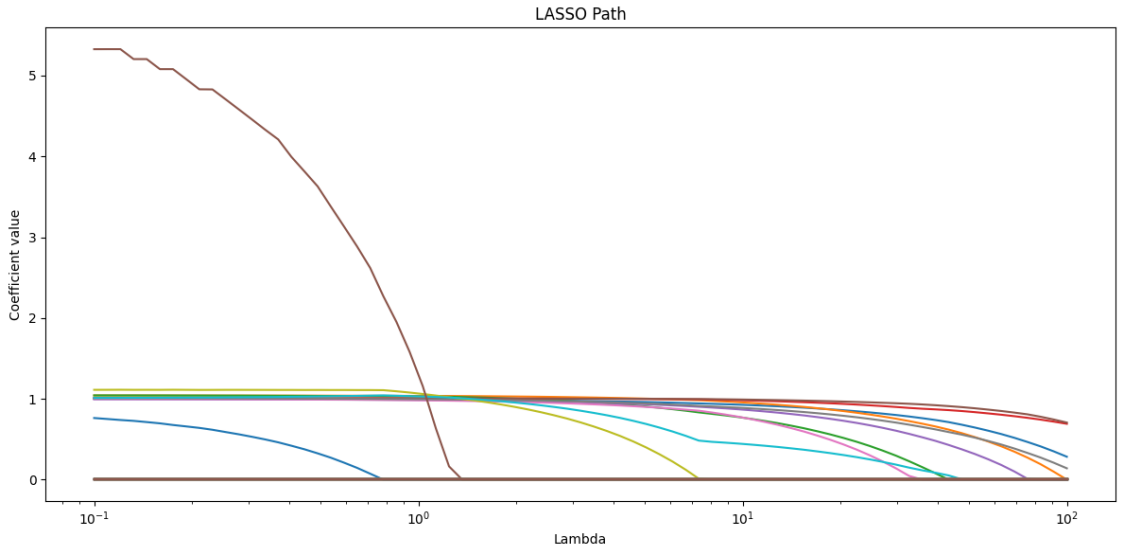

LASSO Path - AUC
AUC
0.65
0.64
0.63
0.62
0.61
0.60
0.59
0.58
10^2
10^1
10^0
10^{-1}
Lambda


LASSO Path - Deviance
Deviance
22500
20000
17500
15000
12500
10000
7500
5000
10^2
10^1
10^0
10^{-1}
Lambda

```
<IPython.core.display.HTML object>
```

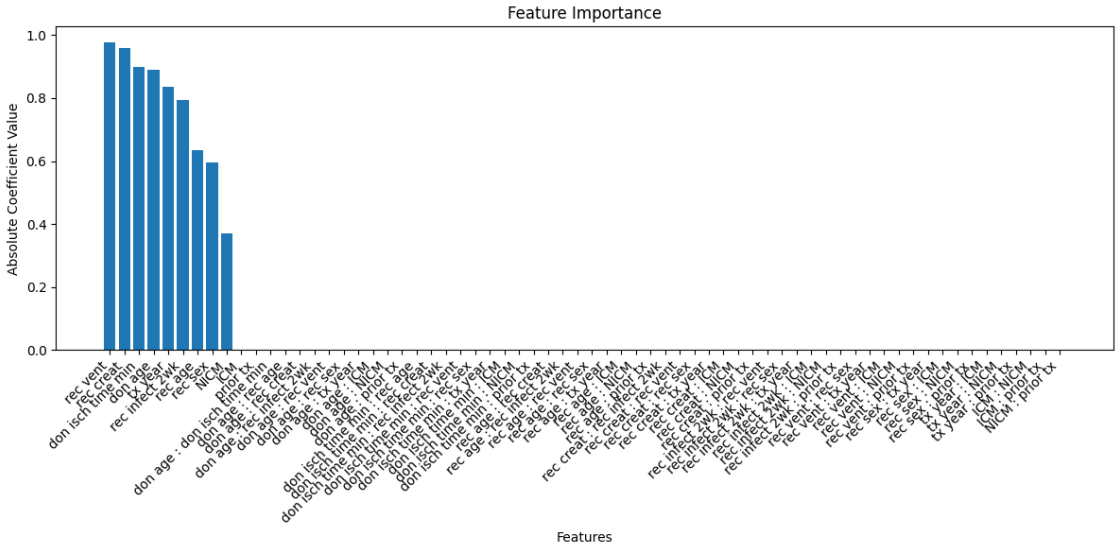
```
All fits converged successfully.
```

[24]:
```
lasso_results_prn.plot_lambda_path()
```



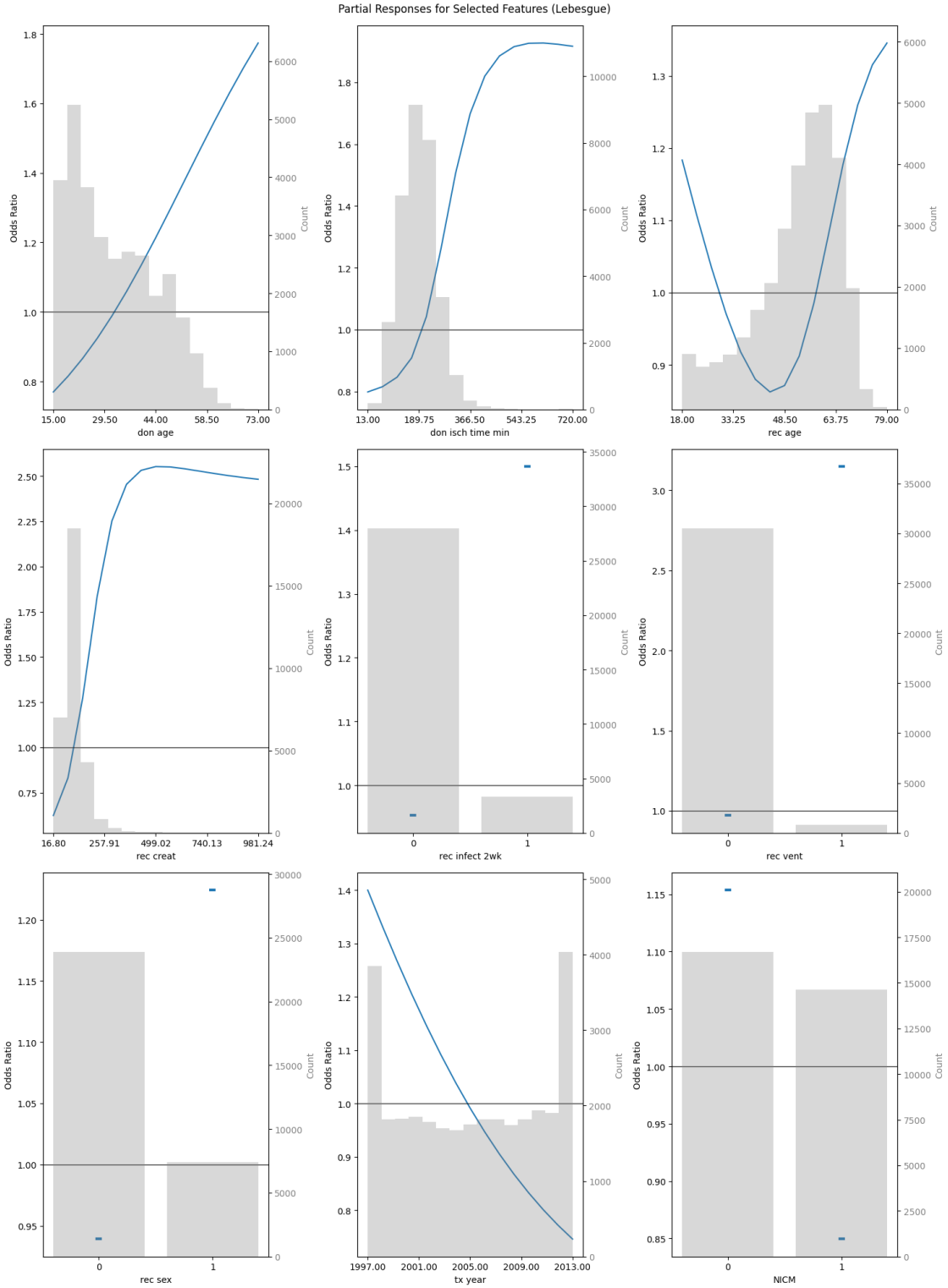
[25]:
```
lasso_results_prn.select_lambda_max_test_auc()

# Alternatively, manually select lambda as needed
# lasso_results_prn.select_lambda(20)

lasso_results_prn.plot_feature_importance()
```

```
Selected lambda index: 20
Selected lambda value: 15.4593
Corresponding test AUC: 0.6042
```



1.10 10. Generate Final Partial Response Plots and Nomogram

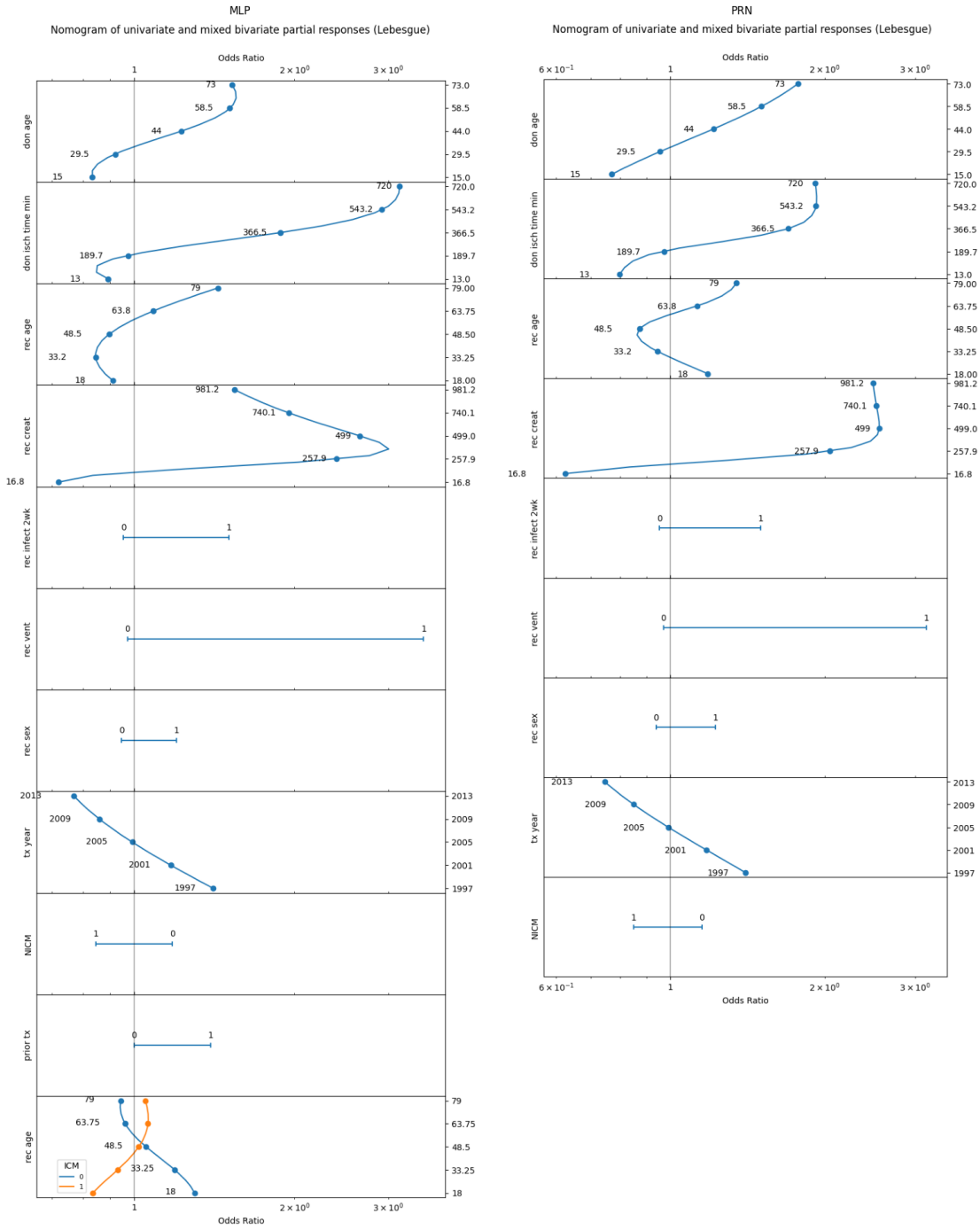
[26]:
```
fig_prn = plot_partial_responses(
    lasso_results_prn,
    X_train_tensor,
    X_train.median().values,
    X_train.std().values,
    partial_response_network,
    n_steps=15,
    sd_scale=2,
    method=partial_response_method,
    device=device,
    categorical_threshold=15,
    subtract_univariate=True,
    figsize=(15,20),
    show_fig=True,
    return_fig=True,
    use_odds_ratio=True
)
```



Finally, we generate a nomogram for our PRN, based on the features selected with LASSO. The nomogram provides a visual tool that can be used to understand the model's decisions.

[27]:
```
%%capture
# ^ Supress nomogram plot output, will display side-by-side instead

nomogram_params_prn = {
    'n_steps': 15,
    'sd_scale': 2,
    'method': partial_response_method,
    'device': device,
    'categorical_threshold': 15,
    'subtract_univariate': True,
    'show_fig': False,
    'return_fig': True,
    'use_odds_ratio': True
}

nomogram_results_prn = nomogram(
    lasso_results_prn,
    X_train_tensor,
    X_train.median().values,
    X_train.std().values,
    partial_response_network,
    **nomogram_params_prn
)

nomogram_main_prn = nomogram_results_prn[6]
nomogram_non_mixed_prn = nomogram_results_prn[7]
```

[28]:
```
display_nomograms_side_by_side(nomogram_main_mlp, nomogram_non_mixed_mlp,
                               nomogram_main_prn, nomogram_non_mixed_prn)
```



1.11 11. Evaluate PRN-LASSO Model

Finally, we evaluate the simplified model (multivariate logistic regression with LASSO regularization) trained on the partial responses of the PRN, based on the selected lambda.

This notebook demonstrates the PRiSM method, transforming a black box model into an inter-

pretable one without sacrificing predictive performance. This approach is particularly valuable when understanding the model's decision-making process is crucial for building trust and gaining insights in clinical decision-making scenarios like predicting one-year mortality after heart transplantation.

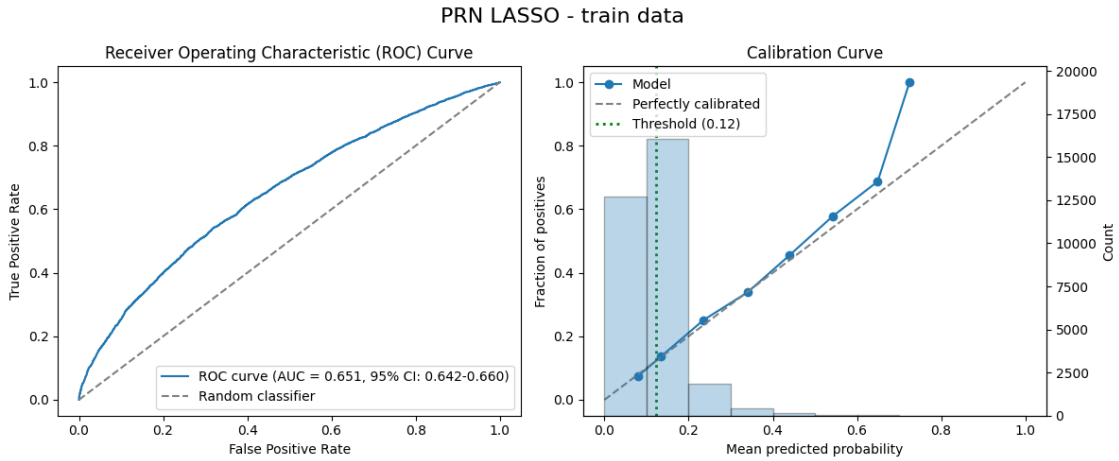
The final PRN-LASSO model provides both high predictive accuracy and interpretability, allowing us to understand how different features contribute to the predictions of post-transplant mortality risk.

[29]:
```
prn_lasso = lasso_results_prn.get_selected_model()
```

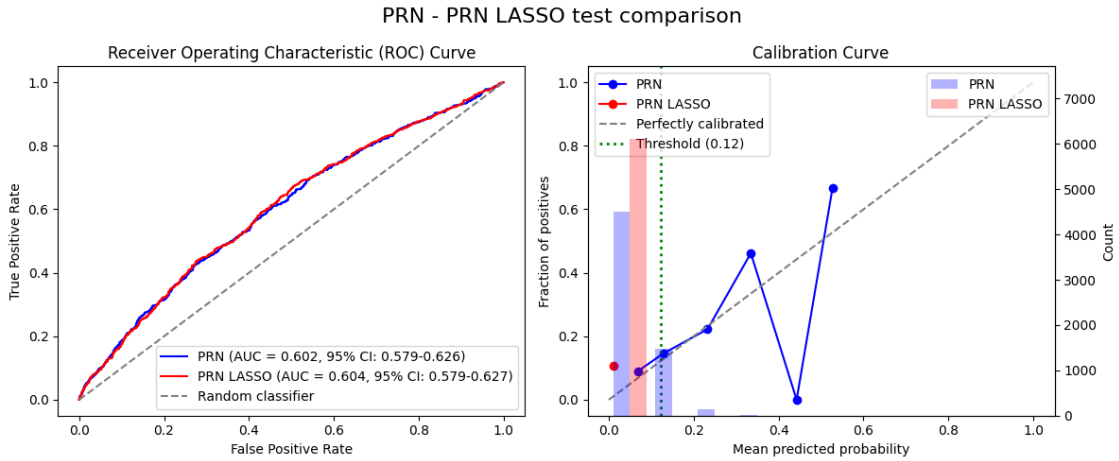
```
Logistic regression model for Lambda index 20 (15.46) selected.
```

[30]:
```
# Evaluate PRN-LASSO on training data
y_pred_train_prn_lasso = prn_lasso.predict_proba(partial_responses_train_prn)[:
 ↪, 1]

results_prn_lasso_train = evaluate_model_performance(y_train,␣
 ↪y_pred_train_prn_lasso, y_train, title="PRN LASSO - train data")

# Evaluate PRN-LASSO on test data and compare to PRN
y_pred_test_prn_lasso = prn_lasso.predict_proba(partial_responses_test_prn)[:,␣
 ↪1]

results_prn_prn_lasso_test = compare_model_performance(y_test, y_pred_test_prn,␣
 ↪y_pred_test_prn_lasso, y_train=y_train, model_names=("PRN","PRN LASSO"),␣
 ↪title="PRN - PRN LASSO test comparison")

# Compare PRN-LASSO and original MLP (blackbox) model
results_prn_mlp_test = compare_model_performance(y_test, y_pred_test_blackbox,␣
 ↪y_pred_test_prn_lasso, y_train=y_train, model_names=("MLP","PRN LASSO"),␣
 ↪title="MLP - PRN LASSO test comparison")
```

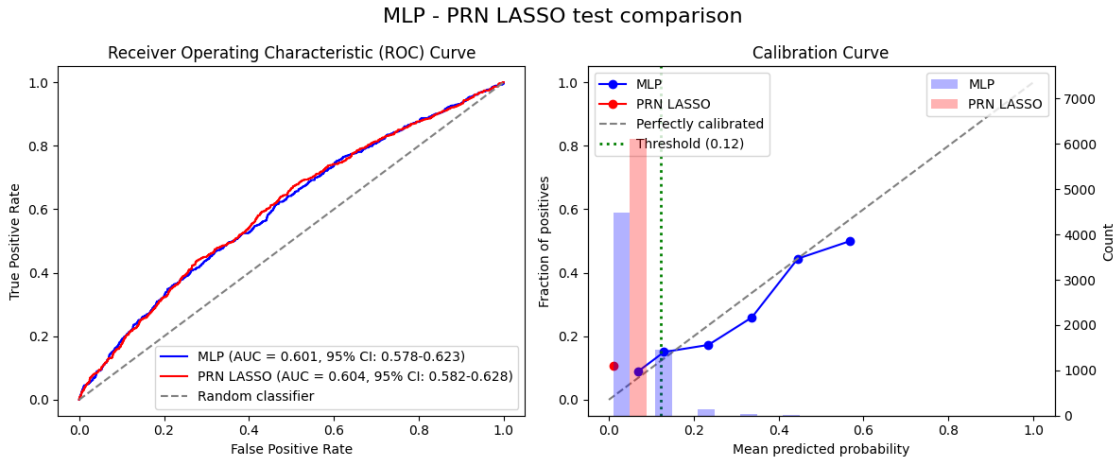
```
---- Model Performance Metrics PRN LASSO - train data ----
Threshold (prevalence) set to: 0.123
AUROC:          0.651 (95% CI: 0.642-0.660) (Area Under the Receiver Operating
Characteristic curve)
Accuracy:       0.643 (Proportion of correct predictions)
Precision:      0.186 (Proportion of true positives among positive predictions,
aka PPV)
Recall:         0.563 (Proportion of true positives among actual positives, aka
sensitivity)
F1 Score:       0.280 (Harmonic mean of precision and recall)
-----------------------------------------------
```




```
-------- Model Performance Comparison --------
Threshold (prevalence) set to: 0.123

PRN:
AUROC:          0.602 (95% CI: 0.579-0.626)
Accuracy:       0.803
Precision:      0.174
Recall:         0.223
F1 Score:       0.195

PRN LASSO:
AUROC:          0.604 (95% CI: 0.579-0.627)
Accuracy:       0.893
Precision:      0.000
Recall:         0.000
F1 Score:       0.000
----------------------------------------------

c:\Users\localuser\PRiSM\prism_github\venv_prism\Lib\site-
packages\sklearn\metrics\_classification.py:1531: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```




```
-------- Model Performance Comparison --------
Threshold (prevalence) set to: 0.123

MLP:
AUROC:          0.601 (95% CI: 0.578-0.623)
Accuracy:       0.797
Precision:      0.175
Recall:         0.241
F1 Score:       0.203

PRN LASSO:
AUROC:          0.604 (95% CI: 0.582-0.628)
Accuracy:       0.893
Precision:      0.000
Recall:         0.000
F1 Score:       0.000
-----------------------------------------------

c:\Users\localuser\PRiSM\prism_github\venv_prism\Lib\site-
packages\sklearn\metrics\_classification.py:1531: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



[31]:
```
# Calculate partial responses on validation dataset
partial_responses_val_prn = partial_responses(X_val_tensor,␣
  ↪partial_response_network, **partial_responses_params_prn)
```

```
Main compute device: cuda
Max threads: 1
Batch size: 256
Univariate 0,   (cuda)
Univariate 1,   (cuda)
Univariate 2,   (cuda)
Univariate 3,   (cuda)
Univariate 4,   (cuda)
Univariate 5,   (cuda)
Univariate 6,   (cuda)
Univariate 7,   (cuda)
Univariate 8,   (cuda)
Univariate 9,   (cuda)
Univariate 10,  (cuda)
Bivariate 0,1,  (cuda)
Bivariate 0,2,  (cuda)
Bivariate 0,3,  (cuda)
Bivariate 0,4,  (cuda)
Bivariate 0,5,  (cuda)
Bivariate 0,6,  (cuda)
Bivariate 0,7,  (cuda)
Bivariate 0,8,  (cuda)
Bivariate 0,9,  (cuda)
Bivariate 0,10, (cuda)
Bivariate 1,2,  (cuda)
Bivariate 1,3,  (cuda)
Bivariate 1,4,  (cuda)
```

```
Bivariate 1,5,  (cuda)
Bivariate 1,6,  (cuda)
Bivariate 1,7,  (cuda)
Bivariate 1,8,  (cuda)
Bivariate 1,9,  (cuda)
Bivariate 1,10, (cuda)
Bivariate 2,3,  (cuda)
Bivariate 2,4,  (cuda)
Bivariate 2,5,  (cuda)
Bivariate 2,6,  (cuda)
Bivariate 2,7,  (cuda)
Bivariate 2,8,  (cuda)
Bivariate 2,9,  (cuda)
Bivariate 2,10, (cuda)
Bivariate 3,4,  (cuda)
Bivariate 3,5,  (cuda)
Bivariate 3,6,  (cuda)
Bivariate 3,7,  (cuda)
Bivariate 3,8,  (cuda)
Bivariate 3,9,  (cuda)
Bivariate 3,10, (cuda)
Bivariate 4,5,  (cuda)
Bivariate 4,6,  (cuda)
Bivariate 4,7,  (cuda)
Bivariate 4,8,  (cuda)
Bivariate 4,9,  (cuda)
Bivariate 4,10, (cuda)
Bivariate 5,6,  (cuda)
Bivariate 5,7,  (cuda)
Bivariate 5,8,  (cuda)
Bivariate 5,9,  (cuda)
Bivariate 5,10, (cuda)
Bivariate 6,7,  (cuda)
Bivariate 6,8,  (cuda)
Bivariate 6,9,  (cuda)
Bivariate 6,10, (cuda)
Bivariate 7,8,  (cuda)
Bivariate 7,9,  (cuda)
Bivariate 7,10, (cuda)
Bivariate 8,9,  (cuda)
Bivariate 8,10, (cuda)
Bivariate 9,10, (cuda)
```

[32]:
```
# Evaluate PRN-LASSO on validation dataset and compare to PRN
y_pred_val_prn = partial_response_network.predict(X_val_tensor, device=device)
y_pred_val_prn_lasso = prn_lasso.predict_proba(partial_responses_val_prn)[:, 1]
```

```
results_prn_prn_lasso_test = compare_model_performance(y_val, y_pred_val_prn,␣
 ↪y_pred_val_prn_lasso, y_train=y_train, model_names=("PRN","PRN LASSO"),␣
 ↪title="PRN - PRN LASSO validation comparison")
```

```
-------- Model Performance Comparison --------
Threshold (prevalence) set to: 0.123

PRN:
AUROC:          0.606 (95% CI: 0.578-0.633)
Accuracy:       0.580
Precision:      0.130
Recall:         0.569
F1 Score:       0.212

PRN LASSO:
AUROC:          0.604 (95% CI: 0.577-0.631)
Accuracy:       0.182
Precision:      0.105
Recall:         0.960
F1 Score:       0.189
----------------------------------------------------
```